

Mémoire de Fin d'Études

Présenté en vue de l'obtention du

DIPLÔME DE LICENCE EN SCIENCES DE L'INFORMATIQUE ET MULTIMÉDIA - LMD

Intitulé

PharmacieConnect
Plateforme web et mobile de gestion des pharmacies de garde



Organisme d'accueil : IT-STORM Consulting

Réalisé par
Teyeb Mohamed

Sous la direction de
Dr. Riheb Naili

Encadrant professionnel : Achraf BCHIRI

Soutenu publiquement le jj/juin/2026 devant le jury composé de :

Président :	Dr.	Prénom NOM	ISIMG
Membre :	Dr.	Prénom NOM	ISIMG
Membre :	Dr.	Prénom NOM	ISIMG

Dédicaces

À la mémoire de mes chers parents,
qui resteront à jamais dans mon cœur.

Leur amour, leurs sacrifices et leurs prières ont été ma plus grande force.
Que Dieu leur accorde Sa miséricorde et les accueille dans Son vaste paradis.

À mes chères sœurs, Manel et Marwa,
pour leur soutien, leur présence et leur affection.
Merci d'avoir été mon refuge et mon soutien après le départ de nos parents.
Votre encouragement m'a donné la force d'avancer et de réaliser ce travail.

À toute ma famille,
pour son amour, sa confiance et ses encouragements constants.
Votre présence, même silencieuse, a toujours été une source de force et de motivation.

À mes amis,
pour leur soutien, leurs conseils et les moments de partage qui m'ont aidé à garder courage tout au
long de ce parcours.

Je dédie ce mémoire avec amour, respect et reconnaissance.

Remerciements / Préface

Je remercie sincèrement mon encadrant académique, Dr. Riheb Naili, pour son suivi et pour les remarques qu'il m'a adressées tout au long de ce projet de fin d'études. Ses conseils m'ont souvent poussé à reprendre certains choix, à mieux justifier la démarche suivie et à présenter PharmacieConnect d'une manière plus claire.

Je remercie également **IT-STORM Consulting**, mon organisme d'accueil, ainsi que mon encadrant professionnel, **Achraf BCHIRI**, pour l'accueil et les orientations donnés pendant le stage externe. Dans ce cadre, j'ai pu relier le développement réalisé à un besoin concret : permettre à un citoyen de trouver plus facilement une pharmacie de garde, surtout lorsque l'information doit être consultée rapidement.

Je remercie l'équipe pédagogique de l'Institut Supérieur d'Informatique et de Multimédia de Gabès pour l'accompagnement assuré durant cette période. Les échanges avec les enseignants m'ont aidé, dans notre cas, à garder un lien entre les choix techniques, les objectifs du mémoire et les exigences de la formation.

Je souhaite aussi exprimer ma reconnaissance aux enseignants qui ont contribué à ma formation. Les notions acquises en développement web, en développement mobile, en bases de données, en sécurité applicative et en génie logiciel ont été utiles à chaque étape du projet, parfois de manière très directe, parfois comme repères pour prendre de meilleures décisions.

Enfin, je remercie ma famille et mes amis pour leur soutien. Leur présence m'a beaucoup aidé à continuer le travail avec régularité, y compris pendant les moments plus difficiles de l'analyse, du développement, des tests et de la rédaction.

Résumé

Ce projet de fin d'études présente la conception et la réalisation de **PharmacieConnect**, une plateforme web et mobile liée aux pharmacies de garde en Tunisie. L'idée de départ est simple, mais elle devient vite délicate dans la pratique : le citoyen doit trouver une information fiable au bon moment, tandis que l'administration doit pouvoir corriger les pharmacies, les plannings de garde et les données associées sans perdre le contrôle sur leur qualité. Dans notre cas, cette contrainte a orienté la solution vers deux usages complémentaires : la consultation mobile et la gestion sécurisée des données.

La partie serveur a été développée avec **FastAPI**. Elle centralise les données, les règles métier, les endpoints REST ainsi que les mécanismes de sécurité basés sur les jetons JWT, les rôles et les périmètres régionaux. Autour de ce backend, un tableau de bord **React/Vite** permet aux utilisateurs autorisés de suivre et modifier les pharmacies, les gardes, les médicaments, les imports CSV, les indicateurs et les journaux d'audit. L'application mobile, réalisée avec **Expo** et **React Native**, reprend les besoins côté citoyen : recherche par texte, consultation selon la date, géolocalisation, catalogue des médicaments et accès à un assistant conversationnel de premiers secours.

La réalisation ne s'est pas limitée aux interfaces. Elle a aussi demandé un travail de modélisation avec **SQLAlchemy**, de validation des entrées, de contrôle d'accès par rôle et de journalisation des actions sensibles. Nous avons séparé les interfaces clientes, les services métier et la couche de persistance afin de garder une architecture plus lisible pendant les tests backend et les corrections. L'assistant de premiers secours utilise, quant à lui, une récupération d'informations hybride. Les réponses restent volontairement prudentes, car certaines situations nécessitent une prise en charge médicale et ne doivent pas être traitées comme de simples questions applicatives.

Mots-clés : pharmacie de garde, FastAPI, React, Vite, Expo, React Native, JWT, API REST, SQLAlchemy, tableau de bord, application mobile, assistant de premiers secours.

Table des figures

2.1	Diagramme de cas d'utilisation détaillé Source : Auteur	16
3.1	Architecture globale de PharmacieConnect Source : Auteur	19
3.2	Diagramme de classes Source : Auteur	22
3.3	Diagramme de séquence : authentification JWT Source : Auteur	23
3.4	Diagramme de séquence : recherche d'une pharmacie Source : Auteur	24
3.5	Diagramme d'activité : import CSV contrôlé Source : Auteur	25
3.6	Diagramme de composants Source : Auteur	26
3.7	Diagramme de déploiement Source : Auteur	26
4.1	Stack technologique de PharmacieConnect – Source : Auteur	33
4.2	Écran de connexion du tableau de bord administrateur – Source : Auteur	39
4.3	Vue du tableau de bord administrateur – Source : Auteur	39
4.4	Vue du registre des pharmacies – Source : Auteur	40
4.5	Vue de gestion des pharmacies et des gardes – Source : Auteur	40
4.6	Vue du calendrier des gardes – Source : Auteur	41
4.7	Vue cartographique du réseau de pharmacies – Source : Auteur	41
4.8	Interface d'import des pharmacies – Source : Auteur	42
4.9	Interface d'import du planning de garde – Source : Auteur	42
4.10	Vue des journaux d'audit – Source : Auteur	43
4.11	Centre de monitoring de la plateforme – Source : Auteur	43
4.12	Connexion mobile – Source : Auteur	44
4.13	Accueil et recherche mobile – Source : Auteur	44
4.14	Carte mobile des pharmacies – Source : Auteur	44
4.15	Calendrier mobile des gardes – Source : Auteur	45
4.16	Catalogue mobile des médicaments – Source : Auteur	45
4.17	Assistant mobile de premiers secours – Source : Auteur	45

Liste des tableaux

1.1	Cadre institutionnel du projet Source : Auteur	5
1.2	Comparaison fonctionnelle des solutions existantes Source : Auteur, à partir de [1, 2, 3, 4, 5]	6
1.3	Organisation des sprints de réalisation Source : Auteur	8
1.4	Outils de gestion de projet utilisés Source : Auteur	9
2.1	Acteurs de la plateforme PharmacieConnect Source : Auteur	11
2.2	Besoins fonctionnels principaux Source : Auteur	11
2.3	Description textuelle du cas d'utilisation Authentification Source : Auteur	13
2.4	Description textuelle du cas d'utilisation Recherche d'une pharmacie Source : Auteur	14
2.5	Description textuelle du cas d'utilisation Consultation des gardes Source : Auteur	14
2.6	Description textuelle du cas d'utilisation Gestion des pharmacies Source : Auteur	14
3.1	Organisation du backend Source : Auteur	20
3.2	Organisation de l'administration web Source : Auteur	20
3.3	Organisation de l'application mobile Source : Auteur	21
3.4	Entités principales du modèle de données Source : Auteur	21
3.5	Justification des choix techniques Source : Auteur	28
4.1	Environnement de développement – Source : Auteur	32
4.2	Technologies utilisées dans le projet – Source : Auteur	32
4.3	Organisation générale du dépôt – Source : Auteur	33
4.4	Statistiques du jeu de données de l'assistant – Source : Auteur	37
4.5	Performances de l'assistant après entraînement – Source : Auteur	37
4.6	Principales intégrations API – Source : Auteur	45
4.7	Variables d'environnement principales – Source : Auteur	46
4.8	Stratégie de validation – Source : Auteur	47
4.9	Résultats de validation exécutés – Source : Auteur	47
4.10	Preuves de lancement et de build – Source : Auteur	48
4.11	Scénarios fonctionnels testés – Source : Auteur	48
4.12	Difficultés rencontrées et solutions adoptées – Source : Auteur	49

Table des matières

Dédicaces	i
Remerciements / Préface	ii
Résumé	iii
Liste des figures	iv
Liste des tableaux	v
Liste des acronymes	viii
Introduction générale	1
Contexte général	1
Problématique	1
Objectifs du projet	2
Méthodologie adoptée	2
Organisation du rapport	2
1 Cadre conceptuel et étude de l'existant	3
1.1 Introduction	4
1.2 Présentation du domaine pharmaceutique	4
1.3 Contexte général	4
1.4 Cadre du stage externe et organisme d'accueil	4
1.5 Problèmes et défis	5
1.6 Étude de l'existant	5
1.7 Analyse du marché et solutions existantes	6
1.8 Critique de l'existant	6
1.9 Solution proposée : PharmacieConnect	6
1.10 Objectifs	7
1.11 Méthodologie de développement	7
1.12 Outils de gestion de projet	8
1.13 Conclusion	9
2 Analyse et spécification des besoins	10
2.1 Introduction	11
2.2 Acteurs	11
2.3 Besoins fonctionnels détaillés	11

2.4	Besoins non fonctionnels	12
2.5	Règles de gestion	13
2.6	Descriptions textuelles des cas d'utilisation	13
2.7	Diagramme de cas d'utilisation	15
2.8	Conclusion	17
3	Conception	18
3.1	Introduction	19
3.2	Architecture globale	19
3.3	Backend FastAPI	19
3.4	Tableau de bord React/Vite	20
3.5	Application mobile Expo/React Native	20
3.6	Modèle de données	21
3.7	Diagramme de classes	22
3.8	Diagrammes de séquence	22
3.9	Diagramme d'activité	24
3.10	Diagramme de composants et de déploiement	25
3.11	Sécurité	27
3.12	Justification des choix techniques	28
3.13	Conclusion	29
4	Réalisation et tests	30
4.1	Introduction	32
4.2	Environnement matériel et logiciel	32
4.3	Technologies utilisées	32
4.4	Organisation du dépôt	33
4.5	Réalisation backend	33
4.6	Réalisation du tableau de bord administrateur	35
4.7	Réalisation application mobile	35
4.8	Réalisation de l'assistant conversationnel de premiers secours	36
4.9	Captures d'écran	38
4.10	Intégration API	45
4.11	Déploiement et configuration	45
4.12	Tests unitaires, API et manuels	46
4.13	Difficultés rencontrées et solutions adoptées	49
4.14	Conclusion	49
	Conclusion générale	50
	Bilan	50
	Résultats obtenus	50
	Limites	50
	Apport personnel	51
	Perspectives	51
	Bibliographie	52

Liste des acronymes

API Application Programming Interface.

BM25 Best Match 25 (Okapi ranking function).

CORS Cross-Origin Resource Sharing.

CSV Comma-Separated Values.

GPS Global Positioning System.

HTTP HyperText Transfer Protocol.

ISIMG Institut Supérieur d'Informatique et de Multimédia de Gabès.

JWT JSON Web Token.

ORM Object-Relational Mapping.

PCT Pharmacie Centrale de Tunisie.

PFE Projet de Fin d'Études.

RBAC Role-Based Access Control.

REST Representational State Transfer.

RTL Right-To-Left.

SMTP Simple Mail Transfer Protocol.

SQL Structured Query Language.

UML Unified Modeling Language.

URL Uniform Resource Locator.

UX User Experience.

Introduction générale

Contexte général

L'accès à une pharmacie de garde dépend de la qualité d'une chaîne d'information complète : identification des pharmacies, préparation des plannings, mise à jour des coordonnées, publication et consultation par le citoyen. En Tunisie, cette information peut être diffusée à travers plusieurs supports, tels que des listes locales, des publications en ligne, des annuaires ou des fichiers tenus par les responsables concernés [6, 2, 5]. Cette dispersion devient particulièrement problématique lorsqu'un utilisateur cherche une pharmacie disponible la nuit, pendant le week-end ou lors d'un jour férié.

Dans ce contexte, la difficulté ne se limite pas à l'absence d'une interface mobile. Elle concerne aussi la production, la correction et le contrôle des données. Une pharmacie mal localisée, une date de garde erronée ou un fichier CSV incomplet peuvent rendre l'information inutilisable au moment où elle est le plus nécessaire. Le projet doit donc traiter la donnée comme un élément métier sensible, depuis son import jusqu'à son affichage dans l'application mobile.

Le projet **PharmacieConnect** a été conçu pour répondre à ce besoin de bout en bout. Il associe une API backend FastAPI, un tableau de bord administrateur React/Vite, une application mobile Expo/React Native et un assistant conversationnel de premiers secours. Le backend centralise les règles de validation, de sécurité, d'audit et d'accès aux données, tandis que les interfaces couvrent deux parcours principaux : l'administration des informations par des utilisateurs autorisés et la consultation rapide par les citoyens. L'assistant complète ce parcours en proposant une première orientation prudente en premiers secours, sans se substituer aux professionnels de santé ni aux services d'urgence.

Problématique

La problématique du projet peut être formulée ainsi : « Comment concevoir une plateforme capable de centraliser les pharmacies, les plannings de garde et les données associées, tout en garantissant une mise à jour contrôlée par des utilisateurs autorisés et une consultation mobile fiable pour le citoyen ? »

Cette question impose plusieurs contraintes concrètes. L'application mobile doit permettre une recherche par texte, par date ou par position GPS. Le tableau de bord doit faciliter les opérations de gestion sans exposer l'ensemble du système à tous les intervenants. Le backend doit vérifier les droits, appliquer les périmètres régionaux, valider les coordonnées et conserver une trace des opérations sensibles, notamment les imports CSV, les modifications et les connexions.

Objectifs du projet

Le projet vise à réaliser une plateforme complète autour des pharmacies, des gardes et du catalogue de médicaments, enrichie par un assistant de premiers secours. Les objectifs retenus sont les suivants :

- concevoir une API backend centralisant les données, les règles métier et les contrôles d'accès ;
- mettre en place une authentification sécurisée avec des jetons JSON Web Token (JWT) ;
- mettre à disposition une interface web pour gérer les pharmacies, les gardes, les médicaments et les utilisateurs autorisés ;
- réaliser une application mobile orientée vers la recherche de pharmacies, la consultation des gardes et l'accès au catalogue de médicaments ;
- intégrer un assistant conversationnel spécialisé en premiers secours fondé sur une récupération d'informations hybride ;
- intégrer une recherche par proximité à partir des coordonnées Global Positioning System (GPS) ;
- prendre en charge l'import de fichiers Comma-Separated Values (CSV) avec validation des données et retour d'erreurs ;
- ajouter des journaux d'audit, des indicateurs et un contrôle d'accès basé sur les rôles ;
- tester les fonctionnalités principales, y compris les accès protégés, les imports, les restrictions régionales, l'audit et l'assistant.

Méthodologie adoptée

Le projet a été conduit en six cycles courts inspirés de Scrum [7], adaptés au contexte d'un PFE réalisé par un seul étudiant. Chaque cycle a été associé à un livrable vérifiable : cadrage et analyse de l'existant, conception technique, backend, tableau de bord administrateur, application mobile, puis intégration et validation finale. Cette progression a permis de relier rapidement les choix de conception à des éléments concrets du dépôt : endpoints FastAPI, services métier, pages React, écrans mobiles et tests.

La modélisation fait appel au formalisme Unified Modeling Language (UML) [8] pour décrire les acteurs, les cas d'utilisation, les entités et les échanges entre composants. Le développement a démarré par le backend, car il porte les contrats d'API, la persistance, la validation et les règles d'accès. Les interfaces web et mobile ont ensuite été raccordées progressivement afin de vérifier les parcours de consultation et d'administration.

Les fonctionnalités à risque – imports CSV, authentification, gestion des droits, recherche géographique et assistant de premiers secours – ont été vérifiées dès leur livraison par des tests backend, des essais manuels et des contrôles de cohérence entre les interfaces.

Organisation du rapport

Le rapport est organisé en quatre chapitres. Le premier présente le contexte du projet, le domaine pharmaceutique, l'étude de l'existant et la solution proposée. Le deuxième précise les acteurs, les besoins fonctionnels et non fonctionnels ainsi que les règles de gestion. Le troisième détaille la conception de la plateforme : architecture globale, modèle de données, diagrammes UML, sécurité et choix techniques. Le quatrième décrit la réalisation du backend, du tableau de bord, de l'application mobile et de l'assistant, puis présente l'intégration, la configuration, les tests et les difficultés rencontrées. Une conclusion générale, une bibliographie et des annexes complètent le document.

Chapitre 1

Cadre conceptuel et étude de l'existant

Sommaire

1.1	Introduction	4
1.2	Présentation du domaine pharmaceutique	4
1.3	Contexte général	4
1.4	Cadre du stage externe et organisme d'accueil	4
1.5	Problèmes et défis	5
1.6	Étude de l'existant	5
1.7	Analyse du marché et solutions existantes	6
1.8	Critique de l'existant	6
1.9	Solution proposée : PharmacieConnect	6
1.10	Objectifs	7
1.11	Méthodologie de développement	7
1.12	Outils de gestion de projet	8
1.12.1	Jira pour la planification et le suivi	8
1.12.2	Bitbucket pour le versionnement du code	8
1.12.3	Intégration Jira–Bitbucket	9
1.13	Conclusion	9

1.1 Introduction

Avant d'aborder les choix techniques, il est nécessaire de replacer PharmacieConnect dans le contexte opérationnel qui a motivé sa réalisation. Ce chapitre part du fonctionnement des gardes pharmaceutiques, présente le cadre du stage externe, puis examine les difficultés rencontrées lorsqu'une information de garde doit être cherchée, corrigée ou publiée rapidement. Il introduit enfin la réponse retenue et la manière dont le projet a été organisé.

1.2 Présentation du domaine pharmaceutique

Dans ce projet, le domaine pharmaceutique a surtout été abordé à travers une question très pratique : comment retrouver une officine disponible lorsque les horaires habituels ne suffisent plus. Les pharmacies de garde participent à cette continuité de service [6, 9], mais leur utilité dépend fortement de la précision des informations publiées. Pour PharmacieConnect, cela ramène le travail à des données concrètes à maîtriser : nom de la pharmacie, adresse, téléphone, gouvernorat, position GPS, date, horaire et type de garde.

La valeur de ces données dépend directement de leur actualité. Une pharmacie mal localisée, une date de garde incorrecte ou un numéro de téléphone obsolète peut empêcher l'utilisateur de trouver rapidement une solution. C'est pourquoi le projet traite la maintenance des données comme une exigence centrale : les imports CSV sont validés, les comptes administratifs sont contrôlés par rôle et les actions sensibles sont enregistrées dans les journaux d'audit.

1.3 Contexte général

PharmacieConnect regroupe les informations liées aux pharmacies, aux plannings de garde, aux médicaments et aux premiers secours. La plateforme distingue deux usages complémentaires. Le citoyen utilise l'application mobile pour chercher une pharmacie, consulter les gardes, accéder au catalogue de médicaments ou poser une question à l'assistant. L'administrateur et les utilisateurs autorisés utilisent le tableau de bord web pour importer, corriger, contrôler et suivre les données diffusées.

Le backend constitue la couche commune entre ces interfaces. Il expose les endpoints publics nécessaires à la consultation mobile et les endpoints protégés utilisés par l'administration. Cette séparation évite que les règles de validation, de rôle et de journalisation soient dupliquées dans les clients.

1.4 Cadre du stage externe et organisme d'accueil

Le projet a été réalisé dans le cadre d'un stage externe au sein de **IT-STORM Consulting**. Le sujet officiel concerne le développement d'une application mobile permettant aux citoyens de consulter les pharmacies de garde en Tunisie. Les principaux besoins portent sur la recherche par géolocalisation ou par saisie manuelle, l'affichage en liste et sur carte, la consultation des coordonnées et l'accès rapide aux informations utiles.

Le stage s'est déroulé du 07/02/2026 au 07/06/2026, sous l'encadrement professionnel de **Achraf BCHIRI**. Ce cadre externe a orienté le projet vers une solution exploitable, structurée autour d'une API REST [10], d'une base de données, d'une application mobile et d'une interface d'administration pour la mise à jour des données.

TABLEAU 1.1 – Cadre institutionnel du projet
Source : Auteur

Élément	Description
Organisme d'accueil	IT-STORM Consulting.
Adresse	12 Rue Jean Pacilly, 91120 Palaiseau, France.
Encadrant professionnel	Achraf BCHIRI.
Période	Du 07/02/2026 au 07/06/2026.
Nature du projet	Projet de fin d'études externe orienté conception et réalisation logicielle.
Domaine	Santé numérique, accès aux pharmacies, gardes et données pharmaceutiques.
Périmètre technique	API backend, tableau de bord administrateur, application mobile et base de données.
Encadrement	Suivi professionnel, suivi académique, revues d'avancement et validation des livrables.

Ce cadre a orienté la démarche vers un prototype complet et vérifiable. La priorité a été donnée aux fonctionnalités essentielles de recherche mobile, à la séparation des responsabilités, à la fiabilité des données et à la traçabilité des actions administratives.

1.5 Problèmes et défis

Les problèmes observés concernent principalement la disponibilité, la fiabilité et l'exploitation des données. Les informations peuvent être publiées sous des formats différents, mises à jour avec retard ou présentées d'une manière peu adaptée à une recherche mobile.

Les défis retenus pour le projet sont les suivants :

- centraliser les informations sur les pharmacies et les gardes ;
- rendre la recherche simple pour un utilisateur mobile ;
- permettre la consultation par date et par proximité ;
- faciliter l'import de volumes de données à travers des fichiers CSV ;
- sécuriser l'administration et limiter les actions selon les rôles ;
- garder une trace des actions sensibles pour améliorer la traçabilité.

1.6 Étude de l'existant

L'étude de l'existant a été orientée par les besoins du projet. Il ne s'agit pas seulement de vérifier si une solution affiche une pharmacie, mais d'évaluer sa capacité à répondre à plusieurs critères : disponibilité des gardes, usage mobile, géolocalisation, fraîcheur des données, workflow de mise à jour, administration, authentification, import de données, auditabilité et couverture tunisienne.

Les listes statiques diffusées sous forme de page web, d'image ou de document restent faciles à partager, mais elles deviennent limitées lorsqu'un utilisateur veut filtrer par gouvernorat, choisir une date précise ou ouvrir l'adresse sur une carte. Elles ne renseignent pas toujours le processus de mise à jour ni l'origine de la correction lorsqu'une donnée change [6, 2].

Les sites spécialisés et les applications mobiles offrent une expérience plus adaptée à la consultation. Cependant, leur efficacité dépend de la qualité de la base de données, de la fréquence de mise à jour et de la présence d'un espace d'administration capable de contrôler les imports, les droits et les corrections. Cette distinction est importante pour PharmacieConnect, car le projet vise à couvrir à la fois la consultation citoyenne et la gestion opérationnelle des données.

1.7 Analyse du marché et solutions existantes

Pour situer PharmacieConnect, la comparaison n'a pas porté uniquement sur l'existence d'une carte ou d'une liste de pharmacies. Nous avons surtout regardé ce que chaque solution permet réellement de faire lorsqu'une donnée doit être cherchée, corrigée ou contrôlée. Les sources institutionnelles, comme le Conseil National de l'Ordre des Pharmaciens de Tunisie, donnent un cadre fiable pour l'organisation de la profession [1]. Les annuaires médicaux, dont Med.tn, rendent certaines informations plus accessibles au public [2]. Les services cartographiques, tels que Google Maps et OpenStreetMap, aident à localiser une adresse, mais ne traitent pas la planification des gardes ni la validation des fichiers importés [3, 4]. L'application Apothicare montre enfin l'intérêt d'un accès mobile aux pharmacies ouvertes ou de garde [5], même si l'enjeu de gestion interne reste à analyser selon les informations disponibles.

La comparaison suivante reste fonctionnelle : elle évalue les capacités observables ou annoncées par les solutions citées. Lorsque l'information n'est pas confirmée par une source publique, la mention doit être vérifiée avant la version finale [1, 2, 3, 4, 5].

TABLEAU 1.2 – Comparaison fonctionnelle des solutions existantes

Source : Auteur, à partir de [1, 2, 3, 4, 5]

Solution	API REST	App mobile	Gestion planning	Auth.	Import données	Couverture
Sources officielles (CNOPT)	Non	Non	Partiel	Non	Non	Oui
Annuaire médicaux (Med.tn)	Non	Non	Partiel	Non	Non	Oui
Cartographie (Google Maps / OSM)	Oui	Oui	Non	Non	Non	Partiel
Apps mobiles (Apothicare)	Non	Oui	Oui	Partiel	Non	Oui
Listes statiques	Non	Non	Non	Non	Non	Partiel
Pharmacie Connect	Oui	Oui	Oui	Oui	Oui	Oui

1.8 Critique de l'existant

L'étude de l'existant montre que le principal risque ne concerne pas uniquement l'affichage des pharmacies. Il concerne aussi la maîtrise de la donnée. Une solution peut permettre la consultation, mais rester limitée si elle ne précise pas comment les plannings sont importés, corrigés, validés et tracés. Pour un utilisateur mobile, cette limite se traduit par une recherche moins fiable. Pour l'administrateur, elle se traduit par une difficulté à savoir qui a modifié une donnée, quand et à partir de quelle source.

Cette difficulté augmente avec le volume de données et le nombre d'intervenants. Une saisie manuelle devient insuffisante lorsque plusieurs assistants régionaux doivent gérer des pharmacies, des gardes et des fichiers CSV. Sans validation des colonnes, contrôle des doublons, restriction des rôles et journalisation, la base peut devenir incohérente sans que l'origine de l'erreur soit identifiable.

1.9 Solution proposée : PharmacieConnect

PharmacieConnect a donc été pensée comme une réponse progressive aux limites observées. Plutôt que de séparer la consultation et la gestion, le projet regroupe les deux dans une même chaîne technique, avec un assistant de premiers secours ajouté au parcours mobile :

- une API backend FastAPI qui centralise les données, gère l'authentification, applique les règles métier et expose les endpoints Representational State Transfer (REST) ;

- un tableau de bord administrateur React/Vite qui offre aux administrateurs un espace de gestion pour les pharmacies, les gardes, les médicaments, les imports CSV, les assistants régionaux, les indicateurs et les journaux d'audit ;
- une application mobile Expo/React Native centrée sur le citoyen, avec la recherche de pharmacies, la consultation des gardes, la géolocalisation, l'accès au catalogue de médicaments et l'assistant de premiers secours.

Cette organisation répond surtout à deux difficultés rencontrées dans l'existant : l'information est parfois accessible, mais sa mise à jour et son contrôle restent peu visibles. Dans notre solution, le backend garde la maîtrise des données et des droits, tandis que les interfaces se concentrent chacune sur son usage : consultation rapide côté citoyen, correction et suivi côté administration. L'assistant ajoute une aide complémentaire, volontairement prudente, sans prétendre remplacer un professionnel de santé ni les services d'urgence.

1.10 Objectifs

Les objectifs fonctionnels et techniques du projet sont :

- offrir une recherche rapide des pharmacies ;
- afficher les pharmacies de garde selon une date donnée ;
- permettre une recherche par proximité géographique ;
- administrer les pharmacies, les gardes, les médicaments et les comptes autorisés ;
- importer des fichiers CSV en contrôlant la qualité des données ;
- sécuriser les accès à travers l'authentification, les rôles et la validation côté serveur ;
- produire des journaux d'audit et des indicateurs utiles à l'administration.

1.11 Méthodologie de développement

Le projet a adopté une organisation par sprints inspirée de Scrum [7], adaptée au contexte d'un PFE. Chaque sprint a été associé à un objectif concret et à des livrables vérifiables. Les arbitrages ont porté sur les modules réellement développés : contrats d'API, imports CSV, authentification, gestion des rôles, pages d'administration, écrans mobiles et tests de validation.

TABLEAU 1.3 – Organisation des sprints de réalisation
Source : Auteur

Sprint	Objectif	Livrables principaux
Sprint 1	Analyse et cadrage	Besoins, acteurs, règles de gestion, étude de l'existant.
Sprint 2	Conception technique	Architecture, modèle de données, sécurité, contrats API.
Sprint 3	Backend	Routes FastAPI, services métier, authentification, imports et audit.
Sprint 4	Administration web	Pages React/Vite, tableaux, formulaires, indicateurs et gestion des droits.
Sprint 5	Application mobile	Recherche, carte, gardes, médicaments, paramètres et langues.
Sprint 6	Intégration et validation	Tests, build, corrections, rapport et annexes.

Cette méthode a limité les risques liés à une intégration tardive. Les modules sensibles, notamment l'authentification, les imports CSV, les restrictions régionales, les journaux d'audit et la recherche mobile, ont été vérifiés progressivement par des tests automatisés ou par des scénarios fonctionnels.

1.12 Outils de gestion de projet

La conduite du projet s'est appuyée sur des outils de suivi et de versionnement afin de relier les besoins, les tâches et le code livré. **Jira** a été utilisé pour organiser les tickets, les sprints et les anomalies [11]. **Bitbucket** a été utilisé pour héberger le dépôt Git, suivre les branches et préparer les fusions de code [12]. Cette organisation a fourni une traçabilité entre les fonctionnalités prévues et les modules effectivement réalisés.

1.12.1 Jira pour la planification et le suivi

Jira a été utilisé pour la planification des sprints et le suivi des tâches. Chaque sprint défini dans le tableau 1.3 a été décliné en tickets, puis en sous-tâches lorsque le module le nécessitait. Les tickets ont notamment couvert les imports CSV, les pages d'administration, les restrictions régionales, les tests et les corrections d'interface.

- un tableau Kanban/Scrum organisé en quatre colonnes : *À faire*, *En cours*, *En révision*, *Terminé* ;
- un suivi des anomalies (*bugs*) et des demandes d'évolution sous forme de tickets Jira, avec attribution d'un niveau de priorité ;
- l'utilisation du *burndown chart* et de la vélocité de sprint pour mesurer l'avancement à chaque cycle.

1.12.2 Bitbucket pour le versionnement du code

Bitbucket a hébergé le dépôt Git du projet. L'organisation du dépôt a suivi une logique inspirée de Gitflow [13] :

- branche `main` pour les versions stables ;

- branche **develop** pour l'intégration courante ;
- branches **feature/*** pour les nouvelles fonctionnalités ;
- branches **bugfix/*** pour les corrections d'anomalies.

Chaque branche de fonctionnalité correspondait à un ticket Jira, en respectant la convention de nommage **feature/PC-XX-description**. Les fusions vers **develop** étaient soumises à une *pull request* faisant l'objet d'une revue de code. Les messages de *commit* référençaient systématiquement le numéro de ticket Jira (par exemple PC-42: ajout import CSV gardes), ce qui a garanti une traçabilité de bout en bout. Le dépôt monorepo regroupait quatre dossiers principaux : **backend/**, **admin/**, **mobile/** et **chatbot/**.

TABLEAU 1.4 – Outils de gestion de projet utilisés
Source : Auteur

Outil	Rôle	Utilisation principale
Jira	Gestion des sprints et des tâches	Planification Scrum, suivi des user stories, gestion des bugs et des évolutions, tableau Kanban et burndown chart.
Bitbucket	Versionnement du code source	Hébergement Git, branches Gitflow, pull requests, revue de code et traçabilité par référence aux tickets Jira.

1.12.3 Intégration Jira–Bitbucket

L'intégration entre Jira et Bitbucket a permis de relier les tickets aux branches, aux commits et aux *pull requests* [14]. Dans le cadre du projet, cette liaison a facilité le suivi entre une exigence fonctionnelle, par exemple l'import des gardes ou la consultation des audit logs, et les modifications de code correspondantes.

1.13 Conclusion

Ce premier chapitre a permis de préciser le contexte du projet, le domaine pharmaceutique et les limites des solutions existantes. Il montre également pourquoi PharmacieConnect repose sur une séparation entre backend, administration web et application mobile. Le chapitre suivant détaille les besoins qui découlent de ce cadrage.

Chapitre 2

Analyse et spécification des besoins

Sommaire

2.1	Introduction	11
2.2	Acteurs	11
2.3	Besoins fonctionnels détaillés	11
2.4	Besoins non fonctionnels	12
2.5	Règles de gestion	13
2.6	Descriptions textuelles des cas d'utilisation	13
2.6.1	Cas d'utilisation : authentification	13
2.6.2	Cas d'utilisation : recherche d'une pharmacie	13
2.6.3	Cas d'utilisation : consultation des pharmacies de garde	14
2.6.4	Cas d'utilisation : gestion des pharmacies	14
2.7	Diagramme de cas d'utilisation	15
2.8	Conclusion	17

2.1 Introduction

Dans ce chapitre, l'analyse est centrée sur les usages concrets de PharmacieConnect : la consultation rapide d'une pharmacie de garde par le citoyen, la mise à jour contrôlée des données par l'administration et la validation des imports CSV. L'objectif est donc de transformer le besoin général d'accès à l'information pharmaceutique en règles exploitables par le backend FastAPI, le tableau de bord React/Vite et l'application mobile Expo/React Native.

2.2 Acteurs

Les acteurs de PharmacieConnect ont été définis à partir des responsabilités réellement présentes dans l'application. Le citoyen consulte les données publiques sans intervenir sur leur contenu, tandis que les comptes d'administration modifient les pharmacies, les gardes et les médicaments à travers des routes protégées. Cette distinction est indispensable, car une erreur sur un planning de garde ou une localisation peut rendre l'information inutilisable au moment de la recherche.

TABLEAU 2.1 – Acteurs de la plateforme PharmacieConnect
Source : Auteur

Acteur	Rôle dans le système
Administrateur	Supervise les pharmacies, les plannings de garde, le catalogue de médicaments, les imports CSV, les indicateurs, les comptes autorisés et les journaux d'audit.
Assistant ou utilisateur autorisé	Met à jour les pharmacies et les gardes dans un périmètre régional limité, selon les permissions accordées par l'administration.
Utilisateur mobile ou citoyen	Recherche une pharmacie, consulte les gardes par date, utilise la géolocalisation, consulte les médicaments et règle ses préférences d'application.
Système	Valide les données, applique les règles métier, sécurise les accès, calcule les distances, journalise les actions et expose les API aux interfaces.

2.3 Besoins fonctionnels détaillés

Les besoins fonctionnels ont été regroupés selon les modules développés dans le dépôt : authentification, pharmacies, gardes, médicaments, administration, application mobile et assistant de premiers secours. Ce découpage correspond aux contrats API utilisés par les clients web et mobile. Il permet également de vérifier que chaque fonctionnalité décrite dans le rapport possède une traduction concrète dans les routes FastAPI, les services métier ou les écrans de l'application.

TABLEAU 2.2 – Besoins fonctionnels principaux
Source : Auteur

Module	Fonctionnalités attendues
Authentification	Connexion, inscription, vérification email, rafraîchissement de session, déconnexion, modification du profil et changement de mot de passe.

Module	Fonctionnalités attendues
Recherche de pharmacies	Recherche par texte, consultation paginée, détail d'une pharmacie, recherche par gouvernorat et recherche par proximité à partir des coordonnées GPS.
Pharmacies de garde	Consultation publique par date, création manuelle, modification, suppression et import CSV des plannings côté administration.
Gestion des pharmacies	Création, modification, suppression, consultation, pagination, filtrage régional et import CSV avec retour des erreurs ligne par ligne.
Gestion des médicaments	Import CSV, mise à jour par code PCT, recherche par nom commercial, DCI ou code PCT, consultation du détail et pagination.
Assistant de premiers secours	Réponse extractive aux questions de premiers secours, récupération hybride, score de confiance, avertissement de sécurité et réponse de repli lorsque la confiance est insuffisante.
Administration	Tableau de bord, gestion des assistants, contrôle des autorisations, carte administrative, indicateurs opérationnels et consultation des journaux d'audit.
Application mobile	Accueil, carte, calendrier des gardes, catalogue de médicaments, paramètres, langues, thème sombre, favoris et notifications.

2.4 Besoins non fonctionnels

Les contraintes non fonctionnelles retenues proviennent directement des risques identifiés dans PharmacieConnect. La sécurité concerne surtout les routes d'administration, les imports CSV et les comptes assistants. La performance est liée à la pagination des pharmacies et des médicaments. La fiabilité dépend de la validation des coordonnées GPS, des dates de garde et des colonnes CSV obligatoires. Ces contraintes orientent donc la conception autant que les fonctionnalités visibles par l'utilisateur.

- **Sécurité** : protéger les routes administratives, hacher les mots de passe, utiliser des jetons JWT, vérifier les rôles, limiter les assistants à leur périmètre régional et tracer les actions sensibles.
- **Performance** : paginer les réponses, limiter les tailles d'import, éviter les réponses trop volumineuses et utiliser le cache lorsque la configuration le permet.
- **Ergonomie** : proposer une application mobile lisible, une administration claire et une navigation adaptée aux opérations fréquentes comme la recherche, l'import et la correction.
- **Maintenabilité** : séparer les routes, services, modèles, schémas, composants et écrans afin de faciliter l'évolution du projet.
- **Disponibilité** : rendre les informations de consultation accessibles à travers des endpoints publics et prévoir une configuration adaptée au déploiement.
- **Fiabilité des données** : valider les fichiers CSV, contrôler les champs obligatoires, signaler les erreurs ligne par ligne et éviter les doublons lorsque des identifiants métier existent.

2.5 Règles de gestion

Les règles de gestion ci-dessous correspondent aux contrôles appliqués côté serveur. Elles ne sont pas laissées aux interfaces clientes, car un appel direct à l'API pourrait contourner une restriction affichée uniquement dans React ou React Native. Le backend vérifie donc les droits, le périmètre régional, la validité des coordonnées, la structure des fichiers importés et la journalisation des opérations sensibles avant toute écriture persistante :

- une route d'administration ne peut être appelée que par un compte authentifié et autorisé ;
- un assistant ne peut intervenir que dans le périmètre régional qui lui est attribué ;
- les mots de passe doivent être hachés avant leur enregistrement ;
- les jetons d'accès expirés doivent être renouvelés à travers un jeton de rafraîchissement valide ;
- une pharmacie doit posséder au minimum un nom et des informations de localisation exploitables ;
- les coordonnées GPS doivent respecter les bornes valides de latitude et de longitude ;
- un planning de garde doit être associé à une pharmacie et à une date valide ;
- un fichier CSV importé doit respecter les colonnes obligatoires du module concerné ;
- les erreurs d'import doivent être retournées avec un message compréhensible ;
- les actions sensibles doivent être journalisées dans les logs d'audit avec un acteur, une action, une cible et un statut.

2.6 Descriptions textuelles des cas d'utilisation

2.6.1 Cas d'utilisation : authentification

TABLEAU 2.3 – Description textuelle du cas d'utilisation Authentification
Source : Auteur

Élément	Description
Acteurs	Administrateur, assistant, utilisateur mobile
Objectif	Donner à l'utilisateur l'accès aux fonctionnalités correspondant à son profil et à ses droits.
Précondition	L'utilisateur possède un compte valide ou suit le processus d'inscription.
Scénario nominal	L'utilisateur saisit ses identifiants depuis le tableau de bord ou l'application mobile. L'API FastAPI vérifie le compte, contrôle son état, enregistre la tentative de connexion, puis émet un jeton d'accès et un jeton de rafraîchissement. Le client conserve la session et n'affiche que les actions autorisées par le rôle retourné.
Postcondition	L'utilisateur est authentifié, une session est créée et seules les fonctionnalités autorisées sont accessibles.
Exceptions	Identifiants invalides, compte désactivé, email non vérifié ou jeton expiré.

2.6.2 Cas d'utilisation : recherche d'une pharmacie

TABLEAU 2.4 – Description textuelle du cas d'utilisation Recherche d'une pharmacie

Source : Auteur

Élément	Description
Acteur	Utilisateur mobile ou citoyen
Objectif	Retrouver une pharmacie à partir d'un mot-clé, d'un gouvernorat ou de la position GPS.
Précondition	L'application mobile est installée et l'API est accessible.
Scénario nominal	L'utilisateur lance une recherche par nom, adresse ou gouvernorat, ou autorise l'utilisation de sa position GPS. L'application mobile transmet les critères à l'API, qui applique les filtres, vérifie les coordonnées disponibles et calcule la distance lorsque la position est fournie. Les résultats sont ensuite retournés avec les informations utiles à la consultation : nom, adresse, téléphone, gouvernorat et position cartographique.
Postcondition	Les pharmacies correspondantes sont affichées sous forme de liste ou sur la carte.
Exceptions	Position indisponible, recherche vide, absence de résultat ou indisponibilité temporaire de l'API.

2.6.3 Cas d'utilisation : consultation des pharmacies de garde

TABLEAU 2.5 – Description textuelle du cas d'utilisation Consultation des gardes

Source : Auteur

Élément	Description
Acteur	Utilisateur mobile ou citoyen
Objectif	Consulter les pharmacies de garde pour une date donnée.
Précondition	Des plannings de garde sont enregistrés dans la base.
Scénario nominal	L'utilisateur choisit une date. L'application interroge l'API des gardes et affiche les pharmacies disponibles avec leurs informations utiles.
Postcondition	Les gardes correspondant à la date choisie sont affichées.
Exceptions	Date invalide, absence de planning ou erreur de communication avec l'API.

2.6.4 Cas d'utilisation : gestion des pharmacies

TABLEAU 2.6 – Description textuelle du cas d'utilisation Gestion des pharmacies

Source : Auteur

Élément	Description
Acteurs	Administrateur, assistant autorisé
Objectif	Ajouter, modifier, consulter ou supprimer les informations des pharmacies.
Précondition	L'acteur est authentifié et possède les droits nécessaires.

Élément	Description
Scénario nominal	L'acteur ouvre la page des pharmacies et effectue l'opération souhaitée. Le backend vérifie les droits, valide les données et enregistre la modification.
Postcondition	Les données sont mises à jour et l'action est journalisée lorsqu'elle est sensible.
Exceptions	Données invalides, pharmacie introuvable, droits insuffisants ou périmètre régional non autorisé.

2.7 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation de PharmacieConnect met en relation les trois zones principales du système : la consultation mobile, la gestion administrative et les traitements backend. Il montre que le citoyen interagit principalement avec les fonctions de recherche, de carte, de calendrier des gardes et de catalogue de médicaments, tandis que l'administrateur et l'assistant régional interviennent sur des opérations contrôlées comme l'import CSV, la correction des pharmacies, la gestion des plannings et la consultation des journaux d'audit. Le diagramme permet ainsi de visualiser la séparation entre les actions publiques et les actions protégées.

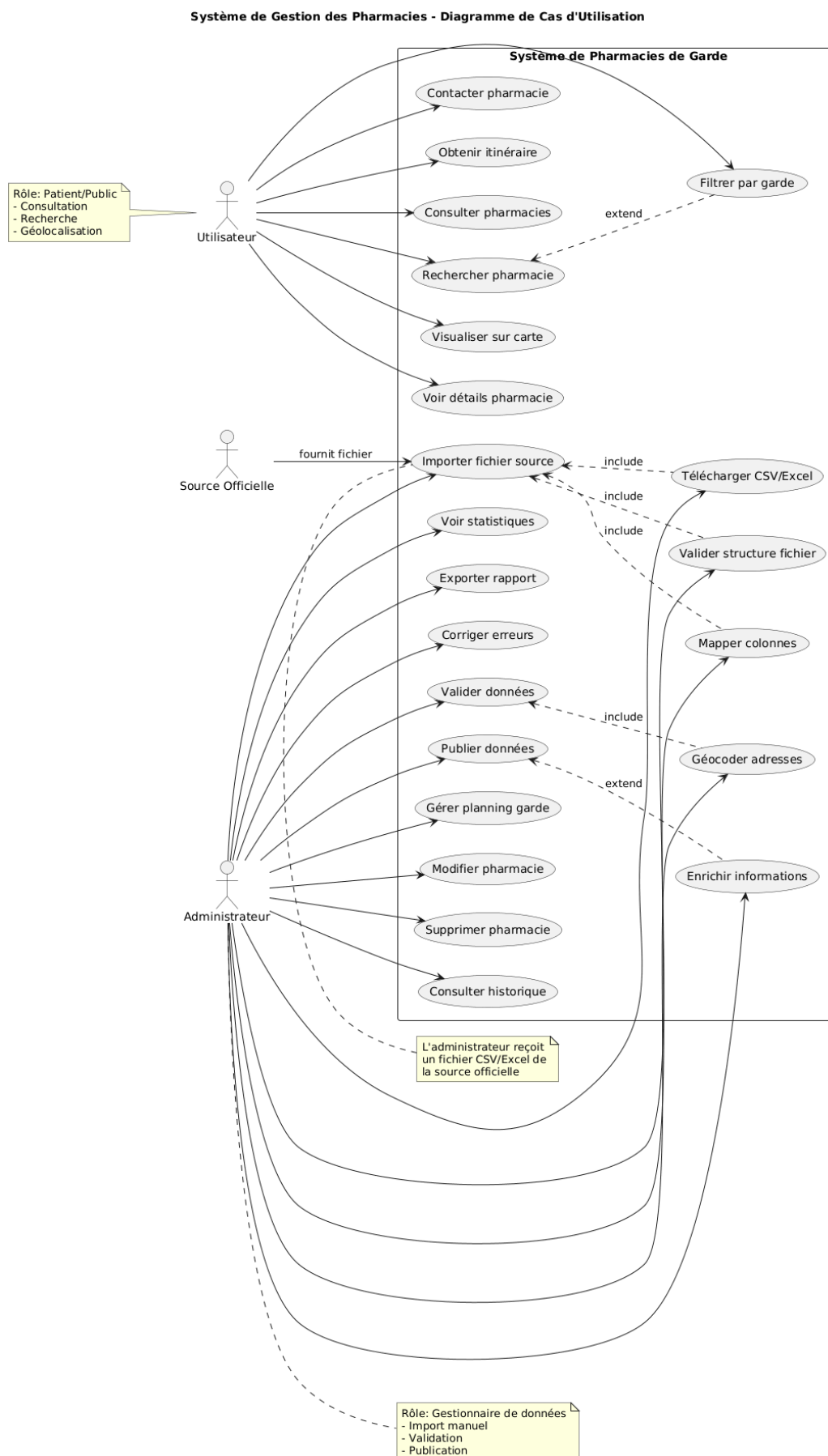


FIGURE 2.1 – Diagramme de cas d'utilisation détaillé
Source : Auteur

2.8 Conclusion

Cette analyse précise les acteurs, les besoins et les règles de gestion de PharmacieConnect. Elle montre que la plateforme doit concilier consultation rapide, administration contrôlée, qualité des données et traçabilité. Le chapitre suivant transforme ces besoins en architecture, modèle de données, diagrammes et mécanismes de sécurité.

Chapitre 3

Conception

Sommaire

3.1	Introduction	19
3.2	Architecture globale	19
3.3	Backend FastAPI	19
3.4	Tableau de bord React/Vite	20
3.5	Application mobile Expo/React Native	20
3.6	Modèle de données	21
3.7	Diagramme de classes	22
3.8	Diagrammes de séquence	22
3.9	Diagramme d'activité	24
3.10	Diagramme de composants et de déploiement	25
3.11	Sécurité	27
3.11.1	Authentification JWT et gestion des sessions	27
3.11.2	Contrôle d'accès basé sur les rôles (Role-Based Access Control (RBAC))	27
3.11.3	Hachage des mots de passe	27
3.11.4	Validation des entrées avec Pydantic	27
3.11.5	Politique Cross-Origin Resource Sharing (CORS) et hôtes autorisés	28
3.11.6	Traçabilité et journalisation des actions	28
3.11.7	Sécurité du déploiement	28
3.12	Justification des choix techniques	28
3.13	Conclusion	29

3.1 Introduction

La conception de PharmacieConnect a consisté à définir comment les besoins fonctionnels décrits précédemment sont répartis entre les différentes parties de la solution. Le backend FastAPI porte les règles métier, la sécurité, les imports et l'accès aux données. Le tableau de bord React/Vite prend en charge les opérations d'administration, tandis que l'application Expo/React Native se concentre sur le parcours citoyen. Cette séparation a été retenue pour éviter la duplication des règles sensibles dans les interfaces et pour garder une architecture plus simple à tester et à faire évoluer.

3.2 Architecture globale

L'architecture de PharmacieConnect place FastAPI comme point de passage unique entre les interfaces et les données. Le tableau de bord React/Vite utilise cette API pour les opérations de gestion, alors que l'application Expo/React Native l'utilise principalement pour la recherche, les gardes, les médicaments et le compte utilisateur. Cette organisation évite que les règles de validation, d'audit et de restriction régionale soient dupliquées dans les clients.

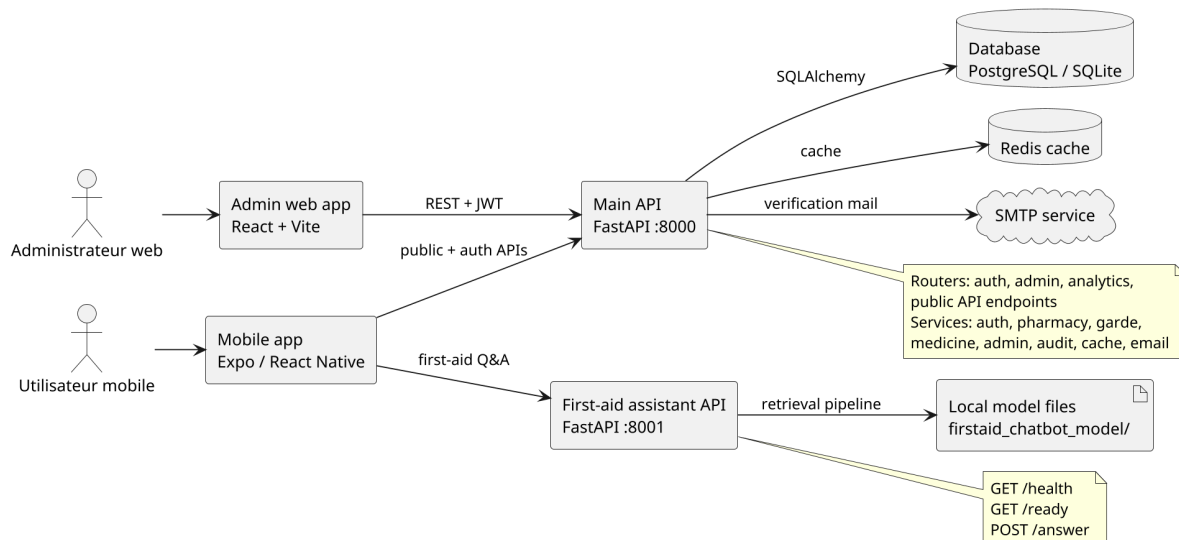


FIGURE 3.1 – Architecture globale de PharmacieConnect

Source : Auteur

L'application mobile utilise principalement les endpoints publics de consultation, les routes liées au compte utilisateur et l'API de l'assistant. Le tableau de bord administrateur s'appuie sur des endpoints protégés pour les opérations de gestion : création, modification, suppression, import CSV, consultation des indicateurs et audit. Le backend devient ainsi la couche commune qui applique les règles métier avant toute lecture ou écriture persistante.

3.3 Backend FastAPI

Dans le projet, FastAPI n'a pas été utilisé uniquement pour exposer des routes REST [15]. Il constitue la couche qui relie les modules essentiels de PharmacieConnect : authentification, pharmacies, gardes, médicaments, imports CSV, audit, indicateurs et assistant de premiers secours. Les routes restent séparées des services métier afin que les règles de validation, les restrictions régionales et les accès à la base via SQLAlchemy soient centralisés dans le backend plutôt que dispersés entre les interfaces.

TABLEAU 3.1 – Organisation du backend
Source : Auteur

Élément	Rôle
<code>main.py</code>	Initialisation de l'application FastAPI, middlewares, migrations au démarrage et endpoints publics.
<code>routers/</code>	Routes d'authentification, d'administration et d'indicateurs.
<code>services/</code>	Logique métier liée à l'authentification, aux pharmacies, aux gardes, aux médicaments, à l'audit et au cache.
<code>models.py</code>	Modèles SQLAlchemy représentant les tables principales.
<code>schemas.py</code>	Schémas Pydantic utilisés pour valider les entrées et structurer les sorties.
<code>migrations/</code>	Scripts SQL permettant de faire évoluer le schéma de la base.
<code>events/</code>	Bus d'événements interne et écouteurs associés.

Le traitement d'une requête suit généralement le même parcours. Le client appelle une route FastAPI; la route vérifie les paramètres et l'utilisateur courant; le service métier applique ensuite les règles nécessaires avant l'accès à la base via SQLAlchemy.

3.4 Tableau de bord React/Vite

Côté administration, React et Vite ont été retenus pour construire une interface que l'on peut faire évoluer page par page [16, 17]. Dans notre cas, cette modularité est utile parce que le tableau de bord ne sert pas seulement à afficher des informations : il doit aussi permettre de corriger les registres, d'importer des fichiers, de vérifier des indicateurs et de suivre les actions sensibles.

TABLEAU 3.2 – Organisation de l'administration web
Source : Auteur

Élément	Rôle
<code>src/pages/</code>	Pages principales du tableau de bord : pharmacies, gardes, médicaments, carte, audit, paramètres et gestion.
<code>src/components/</code>	Composants d'interface, tableaux, formulaires, modales, cartes statistiques et éléments de layout.
<code>src/context/</code>	Gestion de l'authentification, de la langue et de certains états globaux.
<code>src/lib/api.js</code>	Client Axios responsable des appels backend et de l'injection du jeton JWT.
<code>src/styles/</code>	Styles et utilitaires liés à la présentation de l'interface.

Cette organisation évite de mélanger l'affichage avec la logique d'accès aux données. Le client API ajoute le jeton d'accès aux requêtes protégées et prend en charge le renouvellement de session lorsque le jeton expire.

3.5 Application mobile Expo/React Native

Pour la partie mobile, Expo et React Native ont été choisis afin de développer rapidement des écrans centrés sur le parcours citoyen [18, 19]. L'organisation retenue suit les actions que l'utilisateur

réalise réellement dans l'application : chercher une pharmacie, ouvrir la carte, choisir une date de garde, consulter un médicament, gérer son compte ou accéder à l'assistant de premiers secours.

TABLEAU 3.3 – Organisation de l'application mobile
Source : Auteur

Élément	Rôle
<code>screens/</code>	Écrans de l'application : accueil, carte, calendrier, médicaments, connexion, inscription, vérification email et paramètres.
<code>components/</code>	Composants réutilisables comme les cartes de pharmacies, modales, sélecteurs et éléments RTL.
<code>config/api.js</code>	Configuration de l'URL backend et des appels API.
<code>utils/</code>	Fonctions utilitaires pour les données, les couleurs, le thème, le logging et la gestion RTL.
<code>locales/</code>	Fichiers de traduction français, anglais et arabe.

Des contextes React Native partagent certaines informations entre les écrans, comme l'authentification, la langue, le thème, les favoris, les filtres et les notifications. Cette approche évite de répéter la même logique dans plusieurs parties de l'application.

3.6 Modèle de données

Le modèle de données a été construit avec SQLAlchemy [20] à partir des objets réellement manipulés par l'application. Le tableau suivant ne cherche donc pas à donner une définition générale de ces entités ; il précise plutôt comment PharmacieConnect organise les comptes, les pharmacies, les gardes, les médicaments, les sessions, les tentatives de connexion, les recherches et l'audit.

TABLEAU 3.4 – Entités principales du modèle de données
Source : Auteur

Entité	Description
Utilisateur	Compte utilisateur de l'application mobile avec email, mot de passe, statut actif et vérification email.
Administrateur	Compte de gestion avec rôle, statut, informations de profil et périmètre régional éventuel.
RefreshToken	Jeton de rafraîchissement permettant de prolonger une session selon les règles de sécurité.
LoginAttempt	Tentative de connexion utilisée pour suivre les succès et les échecs d'authentification.
Pharmacie	Données d'une pharmacie : nom, adresse, téléphone, gouvernorat, latitude, longitude et référence OSM éventuelle.
GardeSchedule	Planning de garde avec date, pharmacie, horaire, ville, gouvernorat, type de garde et notes.
Medicine	Catalogue de médicaments avec code PCT, nom commercial, prix public, tarif de référence, remboursement, DCI et AP.
AuditLog	Trace des actions importantes avec acteur, cible, statut, adresse IP et détails.
SearchEvent	Événements de recherche utilisés pour alimenter les statistiques et les indicateurs.

3.7 Diagramme de classes

Le diagramme de classes met en évidence le noyau métier de PharmacieConnect. La pharmacie constitue l'entité centrale autour de laquelle sont rattachés les plannings de garde, les coordonnées GPS et les informations de contact. Les comptes administratifs et mobiles sont séparés afin de distinguer la gestion contrôlée de la consultation citoyenne. Les journaux d'audit complètent ce modèle en conservant la trace des opérations sensibles réalisées sur les données.

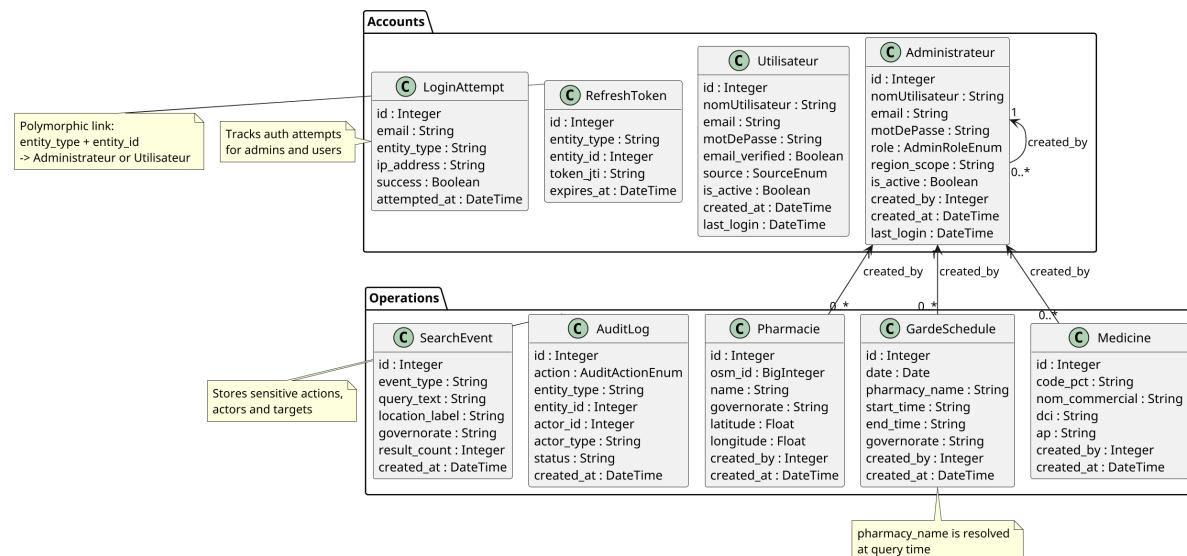


FIGURE 3.2 – Diagramme de classes
Source : Auteur

Le diagramme de la figure 3.2 synthétise les entités centrales du modèle métier de PharmacieConnect et les liens qui les unissent. L'entité **Administrateur** (caractérisée par `email`, `password_hash`, `role` et un éventuel `region_scope`) et l'entité **Utilisateur** (avec `email`, `password_hash`, `email_verified` et `is_active`) représentent les deux profils de comptes manipulés par la plateforme : la gestion côté tableau de bord d'une part, la consultation côté mobile d'autre part. L'entité **Pharmacie** (`nom`, `adresse`, `telephone`, `gouvernorat`, `latitude`, `longitude`) est associée à l'entité **GardeSchedule** (`date`, `horaire`, `type_garde`, `ville`) par une relation un-à-plusieurs : une pharmacie peut figurer dans plusieurs plannings de garde, mais chaque garde référence une seule pharmacie. L'entité **Medicine** (`code_pct`, `nom_commercial`, `dci`, `prix_public`, `tarif_reference`, `remboursement`) reste indépendante du planning de garde et compose le catalogue interrogé par l'application mobile. Enfin, l'entité **AuditLog** (`actor_id`, `action`, `target`, `status`, `ip`, `timestamp`) référence l'**Administrateur** ou l'**Utilisateur** qui a déclenché l'action et conserve une trace horodatée des opérations sensibles (connexions, créations, modifications, suppressions et imports). Cette organisation traduit la séparation des responsabilités voulue par l'architecture : les profils d'accès sont distincts, les données métier sont centralisées autour de la pharmacie et toute action de gestion laisse une trace exploitable côté audit.

3.8 Diagrammes de séquence

Les diagrammes de séquence retenus décrivent deux parcours critiques du projet. Le premier montre comment une session est créée puis contrôlée à travers les jetons JWT. Le second détaille la recherche d'une pharmacie, depuis la saisie ou la géolocalisation côté mobile jusqu'au filtrage effectué par le backend. Ces deux scénarios ont été choisis parce qu'ils concentrent les principaux échanges entre les clients, les services métier et la base de données.

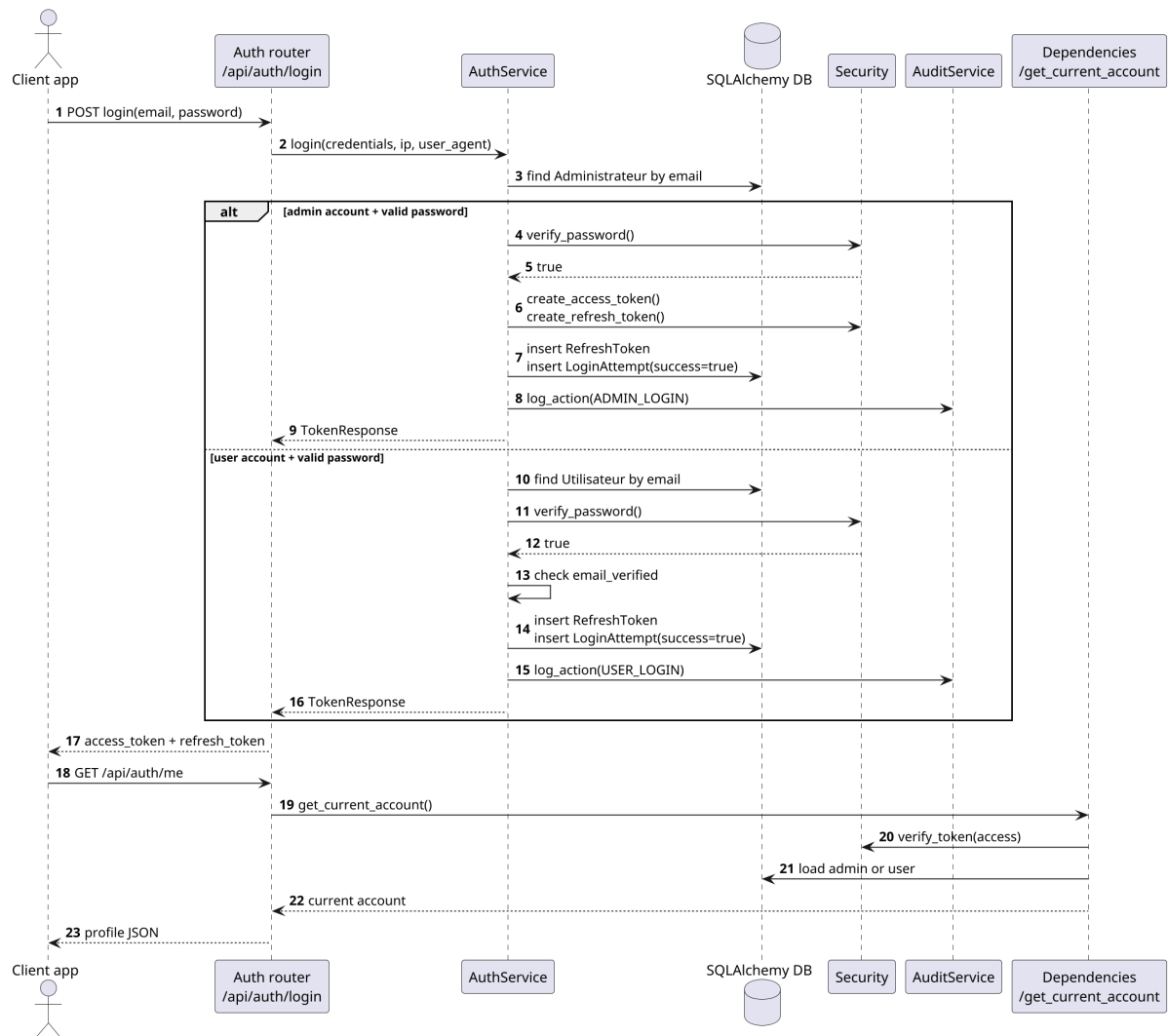


FIGURE 3.3 – Diagramme de séquence : authentification JWT

Source : Auteur

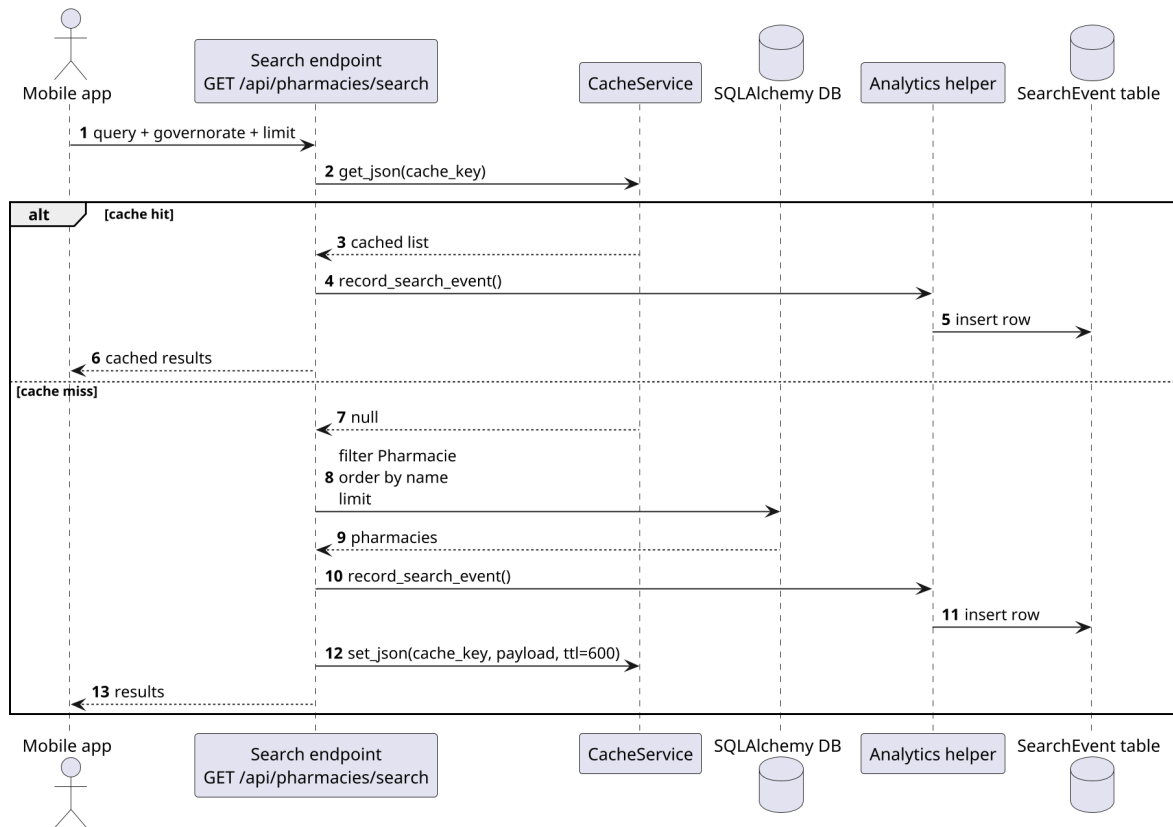


FIGURE 3.4 – Diagramme de séquence : recherche d'une pharmacie
Source : Auteur

3.9 Diagramme d'activité

Le diagramme d'activité détaille le traitement d'un import CSV. Ce scénario est important, car il regroupe plusieurs règles sensibles : contrôle du fichier, validation des colonnes, vérification des lignes, enregistrement des données valides, rejet des lignes incorrectes et journalisation du résultat.

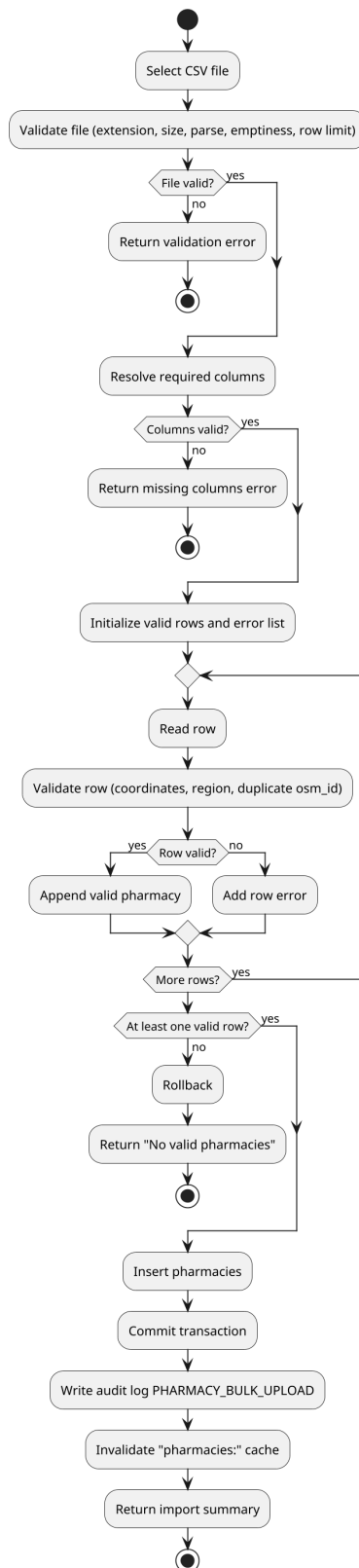


FIGURE 3.5 – Diagramme d'activité : import CSV contrôlé
Source : Auteur

3.10 Diagramme de composants et de déploiement

Les diagrammes de composants et de déploiement complètent la vision technique. Le premier décrit les modules logiciels, tandis que le second présente la répartition logique entre les clients, le

serveur backend, la base de données et la configuration.

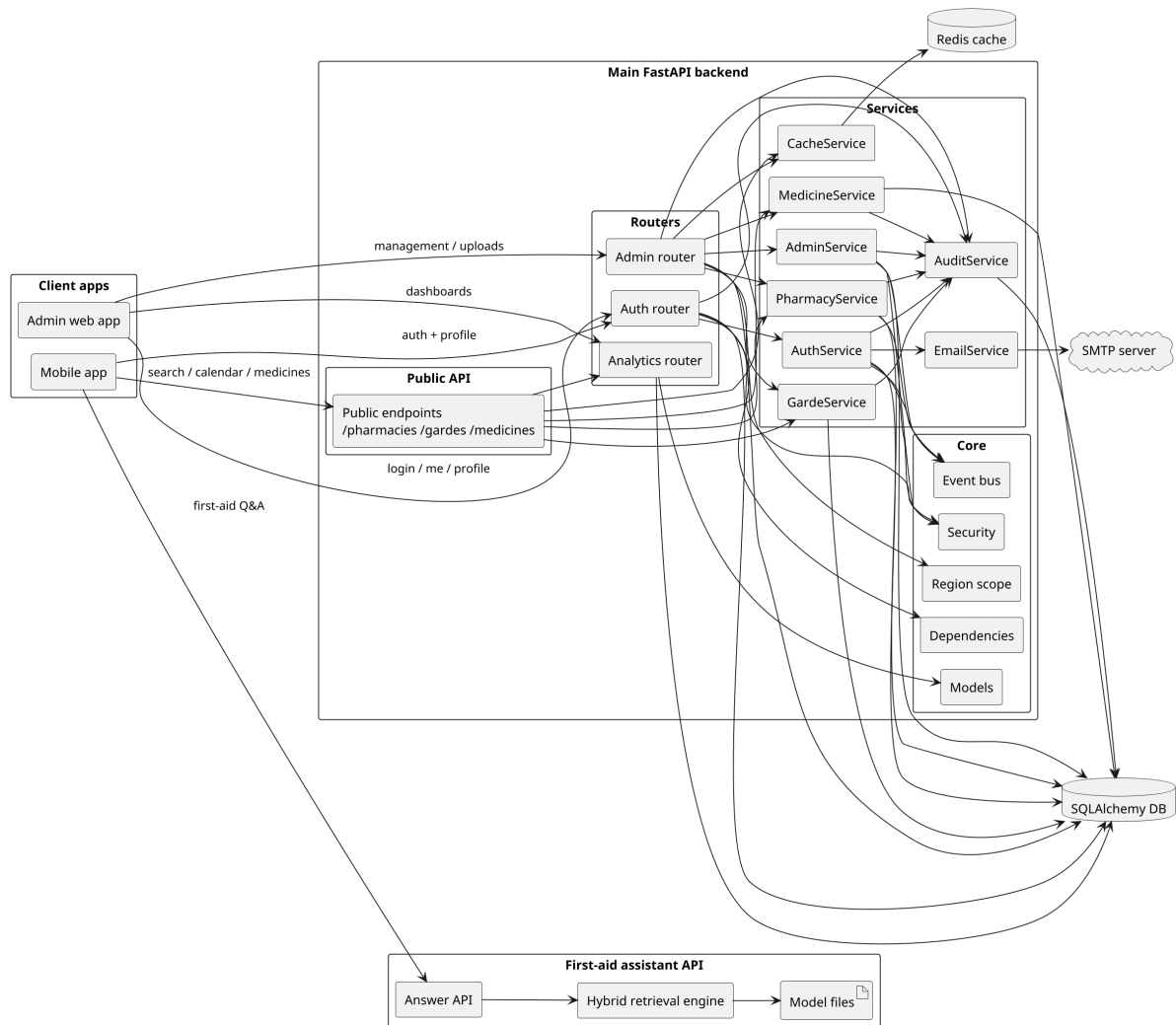


FIGURE 3.6 – Diagramme de composants
Source : Auteur

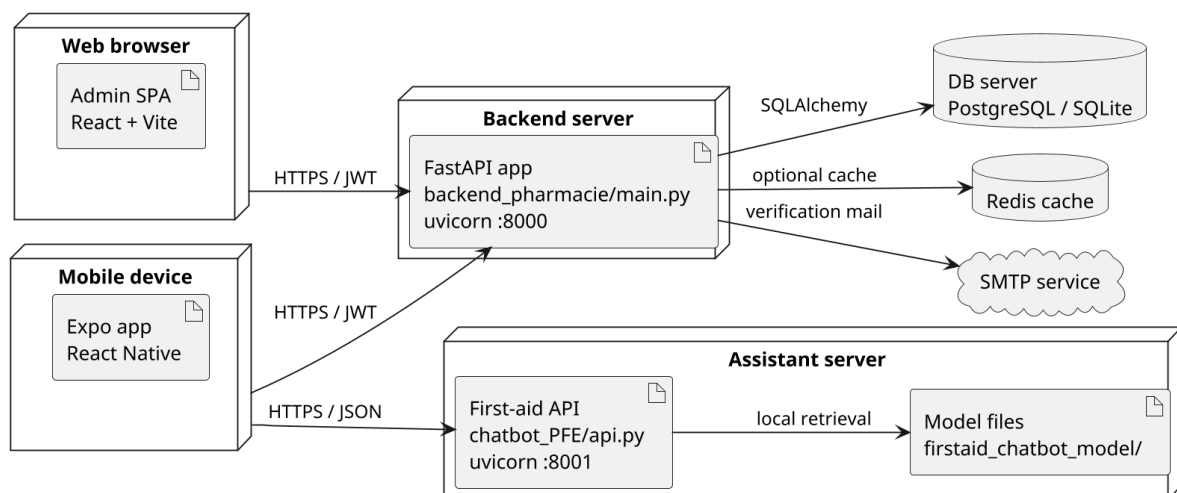


FIGURE 3.7 – Diagramme de déploiement
Source : Auteur

3.11 Sécurité

La sécurité de PharmacieConnect est appliquée côté backend afin que les règles ne dépendent pas uniquement de l’affichage dans les interfaces. Les routes d’administration exigent un jeton JWT valide, puis vérifient le rôle du compte connecté. Pour les assistants, le contrôle ne s’arrête pas au rôle : le périmètre régional est également vérifié avant toute création, modification ou suppression. Cette approche, alignée avec les recommandations OWASP pour la protection des API [21], protège particulièrement les imports CSV, les gardes, les médicaments et les journaux d’audit.

3.11.1 Authentification JWT et gestion des sessions

La gestion des sessions dans PharmacieConnect distingue deux jetons aux durées de vie volontairement différentes. À la connexion, le backend émet un jeton d’accès valable 15 minutes par défaut, signé avec HS256, et un jeton de rafraîchissement valable 7 jours. Le premier accompagne chaque requête protégée dans l’en-tête **Authorization: Bearer**; sa courte durée limite les conséquences d’une interception. Le second est conservé en base dans l’entité **RefreshToken** et effacé lors de la déconnexion explicite, ce qui permet de renouveler la session active sans laisser le jeton d’accès circuler au-delà de sa fenêtre autorisée.

La validation de ces jetons est prise en charge exclusivement par le backend à chaque appel protégé : déléguer ce contrôle à l’interface cliente serait insuffisant, car tout appel direct à l’API contournerait alors les vérifications [22].

3.11.2 Contrôle d’accès basé sur les rôles (RBAC)

PharmacieConnect organise les accès administratifs selon trois rôles : **super_admin**, **admin** et **assistant**. Chaque endpoint protégé évalue le rôle du demandeur avant d’exécuter l’opération. Pour le rôle **assistant**, un deuxième filtre s’applique : l’attribut **region_scope** limite les actions au périmètre régional autorisé. Toute tentative d’opération hors de ce périmètre est rejetée même si le jeton d’accès est valide. Cette combinaison rôle / périmètre répond directement au besoin de délégation régionale : plusieurs assistants peuvent coexister sans modifier les données qui ne relèvent pas de leur zone.

3.11.3 Hachage des mots de passe

Le projet a retenu bcrypt via Passlib pour protéger les mots de passe des administrateurs et des utilisateurs mobiles. Chaque empreinte est produite avec un sel unique généré à l’enregistrement : deux comptes utilisant le même mot de passe donnent des empreintes différentes, ce qui empêche toute comparaison directe entre entrées. Le facteur de coût est réglé pour que la vérification lors de la connexion reste imperceptible pour l’utilisateur, tout en rendant une attaque par volume prohibitivement longue. La comparaison à la connexion s’opère uniquement sur les empreintes stockées ; le mot de passe en clair n’est jamais conservé en base [23].

3.11.4 Validation des entrées avec Pydantic

Avant qu’une donnée entrante n’atteigne la couche service, un schéma Pydantic la valide intégralement. Pour PharmacieConnect, cela concerne notamment les formats d’email des comptes, les plages de coordonnées des pharmacies (latitude bornée entre -90 et 90 , longitude entre -180 et 180), les dates des plannings de garde, les codes PCT des médicaments et la taille maximale des fichiers CSV importés. Regrouper ces contrôles dans les schémas évite de les répliquer dans chaque route et intercepte les données malformées avant tout accès à la base [24].

3.11.5 Politique CORS et hôtes autorisés

Le backend déclare explicitement les origines autorisées via les variables d’environnement de configuration CORS. Toute origine non prévue par la configuration ne doit pas accéder aux routes depuis un navigateur. En complément, `TrustedHostMiddleware` inspecte l’en-tête d’hôte de chaque requête et bloque celles dont l’hôte n’est pas inscrit dans `TRUSTED_HOSTS`. Ces paramètres sont externalisés afin d’adapter la configuration entre le développement local et un déploiement en production.

3.11.6 Traçabilité et journalisation des actions

Deux mécanismes de traçabilité coexistent dans PharmacieConnect. La table `LoginAttempt` enregistre les tentatives de connexion avec leur statut, l’adresse IP source et l’horodatage. En parallèle, le service `AuditService` journalise les opérations administratives significatives : création, modification, suppression d’entités, imports CSV et changements de droits. Chaque entrée d’audit identifie l’acteur, décrit l’opération, référence la cible et indique le résultat. Cette granularité permet de reconstituer l’historique des modifications apportées aux données de la plateforme.

3.11.7 Sécurité du déploiement

Pour un déploiement en production, les mesures complémentaires recommandées sont :

- forcer **HTTPS** sur toutes les communications et rediriger le trafic HTTP ;
- utiliser une `SECRET_KEY` aléatoire d’au moins 256 bits, distincte entre les environnements de développement et de production ;
- remplacer SQLite par **PostgreSQL** pour une gestion robuste des connexions concurrentes ;
- activer le cache **Redis** pour réduire la pression sur la base de données et limiter les requêtes répétitives.

3.12 Justification des choix techniques

TABLEAU 3.5 – Justification des choix techniques
Source : Auteur

Choix	Justification
FastAPI	Retenu pour concentrer les endpoints publics et protégés dans un backend testable, avec validation des requêtes et documentation API utile pendant les essais [15].
React et Vite	Retenus pour structurer le tableau de bord par pages fonctionnelles : pharmacies, gardes, médicaments, audit et paramètres [16, 17].
Expo et React Native	Retenus pour réaliser les écrans mobiles liés au parcours citoyen : recherche, carte, calendrier, médicaments, profil et assistant [18, 19].
SQLAlchemy	Retenu pour représenter les entités métier du projet et garder l’accès aux tables dans les services backend [20].
SQLite et PostgreSQL	SQLite facilite la validation locale, tandis que PostgreSQL reste plus adapté à un déploiement avec connexions concurrentes [25].

Choix	Justification
JWT	Retenu pour séparer un jeton d'accès court et un jeton de rafraîchissement, afin de protéger les routes administratives et les comptes utilisateurs [22].
CSV	Retenu comme format d'import pragmatique pour les pharmacies, les gardes et les médicaments, avec validation des colonnes et retour des erreurs.

3.13 Conclusion

Cette conception fournit une base cohérente pour la réalisation de PharmacieConnect. Les règles sensibles restent dans le backend, les interfaces se concentrent sur leurs parcours respectifs et les données métier sont structurées autour des pharmacies, des gardes, des médicaments et de l'audit. Cette séparation rend les évolutions plus simples à maîtriser, par exemple l'ajout d'une nouvelle règle d'import, d'un rôle administratif ou d'une amélioration de l'assistant de premiers secours. Le chapitre suivant passe de cette conception à la réalisation technique et à la validation de la solution.

Réalisation et tests

Sommaire

4.1	Introduction	32
4.2	Environnement matériel et logiciel	32
4.3	Technologies utilisées	32
4.4	Organisation du dépôt	33
4.5	Réalisation backend	33
4.5.1	Endpoints publics	34
4.5.2	Authentification et sessions	34
4.5.3	Services métier	34
4.5.4	Imports CSV	35
4.5.5	Audit et indicateurs	35
4.6	Réalisation du tableau de bord administrateur	35
4.6.1	Pages principales	35
4.6.2	Gestion des sessions côté administration	35
4.7	Réalisation application mobile	35
4.7.1	Recherche et détail des pharmacies	35
4.7.2	Carte et géolocalisation	36
4.7.3	Calendrier des gardes	36
4.7.4	Catalogue de médicaments	36
4.7.5	Paramètres, langues et thème	36
4.8	Réalisation de l'assistant conversationnel de premiers secours	36
4.8.1	Architecture générale	36
4.8.2	Données d'apprentissage et nettoyage	37
4.8.3	Entraînement et métriques de performance	37
4.8.4	API et intégration avec l'application mobile	38
4.8.5	Tests et validation	38
4.9	Captures d'écran	38
4.10	Intégration API	45
4.11	Déploiement et configuration	45
4.12	Tests unitaires, API et manuels	46
4.12.1	Stratégie de test	46
4.12.2	Résultats de validation automatisée	47
4.12.3	Tests automatisés du backend	47
4.12.4	Tests API	48
4.12.5	Validation du déploiement local	48
4.12.6	Validation fonctionnelle	48

4.13 Difficultés rencontrées et solutions adoptées	49
4.14 Conclusion	49

4.1 Introduction

Après la phase de conception, ce chapitre présente le passage de PharmacieConnect vers une version exécutable et testable. Il décrit l’environnement de développement, les choix techniques réellement mobilisés, l’organisation du dépôt et les principales fonctionnalités implémentées dans le backend, le tableau de bord administrateur, l’application mobile et l’assistant de premiers secours. L’objectif n’est pas seulement d’énumérer les outils utilisés, mais de montrer comment chaque composant contribue aux besoins identifiés : consultation des gardes, gestion administrative, import CSV, traçabilité, géolocalisation et validation des accès.

4.2 Environnement matériel et logiciel

Le développement de PharmacieConnect a mobilisé plusieurs outils, car le projet combine un backend, une interface web d’administration et une application mobile.

TABLEAU 4.1 – Environnement de développement – Source : Auteur

Élément	Description
Machine de développement	Ordinateur portable Intel Core i7-8750H, 12 threads, 16 Go RAM, stockage local 343 Go.
Système d’exploitation	Linux 6.17 x86_64.
Éditeur de code	Visual Studio Code.
Backend	Python, FastAPI, Uvicorn, SQLAlchemy, Pydantic, Pytest
Administration web	Node.js, React, Vite, React Router, Axios, Tailwind CSS
Application mobile	Expo, React Native, React Navigation, i18next
Base de données	SQLite en développement local, PostgreSQL selon la configuration
Outils de test	Pytest, Jest, build Vite et tests fonctionnels manuels

4.3 Technologies utilisées

Les choix techniques ont été arrêtés à partir des responsabilités observées pendant la réalisation. Le backend devait rester le point de contrôle des routes REST, des accès et des règles métier. L’administration web devait rendre les opérations répétées assez rapides pour corriger des pharmacies, des gardes, des imports ou des journaux d’audit. L’application mobile, de son côté, devait garder un parcours court : chercher, filtrer, localiser et consulter l’information utile sans multiplier les écrans.

TABLEAU 4.2 – Technologies utilisées dans le projet – Source : Auteur

Technologie	Utilisation
FastAPI	Exposition des routes REST publiques et protégées du backend [15].
SQLAlchemy	Modélisation des entités persistantes et accès aux données [20].
Pydantic	Validation des entrées et structuration des réponses de l’API [24].
JWT	Gestion des jetons d’accès et de rafraîchissement [22].
React	Construction de l’interface d’administration [16].

Technologie	Utilisation
Vite	Développement local et génération du build du tableau de bord [17].
Axios	Centralisation des appels HTTP entre le tableau de bord et l'API [26].
Tailwind CSS	Mise en forme des écrans d'administration [27].
Expo	Exécution et validation de l'application mobile [18].
React Native	Création des écrans mobiles multiplateformes [19].
React Navigation	Navigation entre les écrans mobiles [28].
i18next	Gestion des langues et des ressources de traduction [29].
OpenStreetMap / Nominatim	Référence cartographique et géocodage lorsque l'enrichissement des coordonnées est nécessaire [30].
BM25 (Okapi)	Récupération lexicale utilisée dans l'assistant de premiers secours [31].
Sentence Transformers	Embeddings sémantiques avec <code>all-MiniLM-L6-v2</code> pour la récupération dense [32].
Retrieval Pipeline	Pipeline hybride de fusion lexicale/sémantique avec reclassement et extraction de réponses.
Pytest	Tests automatisés du backend et de l'assistant [33].
Jest	Tests automatisés côté application mobile [34].

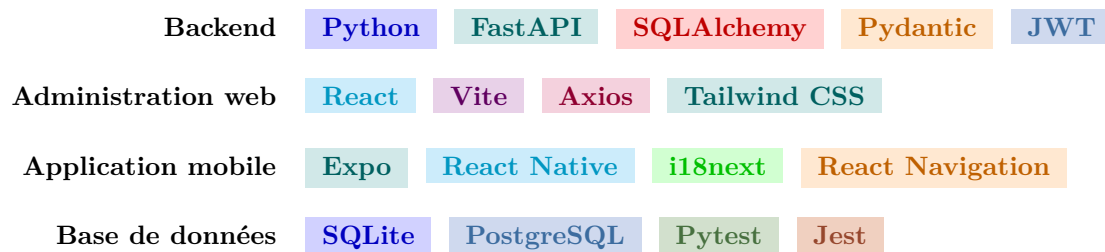


FIGURE 4.1 – Stack technologique de PharmacieConnect – Source : Auteur

4.4 Organisation du dépôt

Le projet est regroupé dans un même espace de travail. Cette structure a facilité l'intégration entre l'API, le tableau de bord et l'application mobile, notamment lors des tests de communication entre les clients et le backend.

TABLEAU 4.3 – Organisation générale du dépôt – Source : Auteur

Dossier	Contenu
backend	API FastAPI, modèles, services, migrations
admin	Tableau de bord React/Vite
mobile	Application mobile Expo/React Native
chatbot	Assistant avec pipeline hybride
docs	Documentation technique

4.5 Réalisation backend

Dans la réalisation, le backend est devenu le point de passage obligatoire entre les interfaces et la base de données. Cette position lui permet de recevoir les requêtes web et mobiles, d'appliquer

les règles métier et de garder au même endroit l'authentification, la validation, l'audit et les imports.

4.5.1 Endpoints publics

Les endpoints publics répondent surtout aux besoins de l'application mobile. Ils donnent accès aux informations de consultation sans passer par l'espace d'administration.

- GET `/health` pour vérifier l'état du backend ;
- GET `/api/pharmacies` pour consulter les pharmacies avec pagination ;
- GET `/api/pharmacies/search` pour rechercher par texte ;
- GET `/api/pharmacies/nearby` pour rechercher les pharmacies proches d'une position GPS ;
- GET `/api/pharmacies/{pharmacy_id}` pour consulter le détail d'une pharmacie ;
- GET `/api/gardes` pour consulter les gardes d'une date donnée ;
- GET `/api/medicines` pour rechercher dans le catalogue de médicaments ;
- GET `/api/medicines/{code_pct}` pour consulter le détail d'un médicament.

Pour la recherche par proximité, l'API utilise les coordonnées GPS envoyées par le client. Elle calcule la distance avec les pharmacies disponibles et retourne les résultats triés par distance.

4.5.2 Authentification et sessions

L'authentification est gérée côté backend afin de garder le contrôle sur les comptes et les autorisations. Lorsqu'une connexion réussit, le serveur retourne un jeton d'accès et un jeton de rafraîchissement.

Listing 4.1 – Structure simplifiée d'une réponse de connexion

```
{
  "access_token": "[jeton d'accès]",
  "refresh_token": "[jeton de rafraîchissement]",
  "token_type": "bearer",
  "expires_in": 900
}
```

Les routes protégées vérifient ensuite le jeton reçu et le rôle de l'utilisateur. Cette vérification est réalisée côté serveur afin qu'un simple contrôle d'interface ne puisse pas être contourné.

4.5.3 Services métier

La logique principale est placée dans des services. Ce choix évite de concentrer trop de code dans les routes et rend les traitements plus faciles à tester.

- `AuthService` pour l'inscription, la connexion, la vérification email, le rafraîchissement et la déconnexion ;
- `PharmacyService` pour la gestion et l'import des pharmacies ;
- `GardeService` pour la gestion et l'import des plannings de garde ;
- `MedicineService` pour l'import, la recherche et la consultation des médicaments ;
- `AuditService` pour la journalisation des actions importantes ;
- `CacheService` pour l'utilisation du cache lorsque la configuration le permet.

4.5.4 Imports CSV

Les imports CSV ont été ajoutés pour éviter la saisie manuelle d'un grand nombre de lignes. Avant l'enregistrement, le backend vérifie l'extension, la taille, les colonnes obligatoires, le format des valeurs et les contraintes métier.

Lorsqu'une ligne est incorrecte, la réponse indique les informations nécessaires pour corriger le fichier. Pour les médicaments, le code PCT sert d'identifiant pour mettre à jour un enregistrement existant au lieu de créer un doublon.

4.5.5 Audit et indicateurs

Les actions sensibles sont enregistrées dans les journaux d'audit. Cela concerne notamment les connexions, les échecs de connexion, les inscriptions, les créations administratives et les imports. Les données statistiques alimentent le tableau de bord et donnent une vision synthétique de l'activité.

4.6 Réalisation du tableau de bord administrateur

Le tableau de bord administrateur rassemble les fonctions de gestion dans une interface web destinée aux administrateurs et aux utilisateurs autorisés.

4.6.1 Pages principales

Les principales pages développées sont :

- le tableau de bord pour les compteurs, indicateurs et activités récentes ;
- la page des pharmacies pour consulter, créer, modifier, supprimer et importer les pharmacies ;
- la page des gardes pour gérer les plannings de garde ;
- la page des médicaments pour importer et consulter le catalogue ;
- la carte pour visualiser les pharmacies géographiquement ;
- la page de management pour gérer les assistants et les autorisations ;
- la page des journaux d'audit pour consulter les actions enregistrées ;
- les pages de profil, paramètres, notifications et langues.

4.6.2 Gestion des sessions côté administration

Le tableau de bord utilise un client API centralisé. Ce client ajoute le jeton JWT aux requêtes protégées. Si le jeton d'accès expire, il tente de renouveler la session. En cas d'échec, l'utilisateur est redirigé vers la page de connexion.

4.7 Réalisation application mobile

L'application mobile correspond au parcours le plus visible pour le citoyen. Nous l'avons organisée autour des usages les plus directs : rechercher une pharmacie, consulter les gardes, ouvrir la carte, parcourir le catalogue de médicaments et ajuster quelques paramètres.

4.7.1 Recherche et détail des pharmacies

Depuis l'écran d'accueil, l'utilisateur peut rechercher une pharmacie par texte et consulter les résultats. La fiche de détail affiche les informations disponibles, comme le nom, l'adresse, le téléphone, le gouvernorat et les coordonnées lorsque celles-ci existent.

4.7.2 Carte et géolocalisation

L'application peut demander l'autorisation d'utiliser la position de l'utilisateur. Cette position est envoyée au backend afin de récupérer les pharmacies proches. La carte affiche ensuite les résultats sous forme de marqueurs.

4.7.3 Calendrier des gardes

Le calendrier donne à l'utilisateur le choix d'une date et affiche les pharmacies de garde correspondantes. Cette fonctionnalité répond au besoin principal du projet : trouver rapidement une pharmacie disponible à une date précise.

4.7.4 Catalogue de médicaments

L'écran des médicaments prend en charge la recherche par nom commercial, DCI ou code PCT. La fiche de détail affiche les informations présentes dans la base, comme le prix public, le tarif de référence et la catégorie de remboursement.

4.7.5 Paramètres, langues et thème

L'application prend en charge le français, l'anglais et l'arabe. Le choix de l'arabe active l'affichage RTL. Les paramètres regroupent aussi le thème, les notifications, les favoris et certaines préférences utilisateur.

4.8 Réalisation de l'assistant conversationnel de premiers secours

L'assistant conversationnel complète le parcours principal de PharmacieConnect en proposant une aide de premiers secours à côté de la recherche de pharmacies de garde. Pour limiter le risque de réponses non ancrées dans les données disponibles, l'implémentation privilégie une architecture de récupération d'informations hybride : une recherche lexicale par BM25 [31] et une recherche sémantique fondée sur des embeddings `all-MiniLM-L6-v2` [32]. Les réponses sont donc construites à partir de passages récupérés, puis accompagnées d'un score de confiance et d'avertissements pour les situations critiques.

4.8.1 Architecture générale

L'architecture de l'assistant suit un pipeline en plusieurs étapes :

1. **Analyse de la requête** : Normalisation, tokenisation, détection de type et sujet.
2. **Récupération parallèle** : Deux algorithmes complémentaires :
 - **BM25 (Okapi)** : récupération lexicale sensible aux termes de la question ;
 - **Embeddings sémantiques** : représentation vectorielle des questions et passages avec `all-MiniLM-L6-v2`.
3. **Fusion hybride** : Normalisation et fusion des scores :
 - 0.55 pour BM25 (lexical) ;
 - 0.35 pour embeddings sémantiques ;
 - 0.10 pour bonus de correspondance thématique.
4. **Reclassement et extraction** : reclassement par similarité, alignement thématique et couverture, puis réponse extractive composée du texte récupéré.
5. **Contrôle de confiance** : calcul multi-signaux (score, similarité, cohérence) et réponse de fallback lorsque la confiance est insuffisante.
6. **Sécurité médicale** : Avertissement auto pour sujets à risque (étouffement, choc, hémorragie).

4.8.2 Données d'apprentissage et nettoyage

Le jeu de données de départ s'appuie sur FirstAidQA, un corpus synthétique orienté premiers secours et situations d'urgence [35]. Dans PharmacieConnect, ce corpus a été nettoyé, segmenté et complété par des exemples ciblés afin d'obtenir un index mieux adapté aux questions courtes posées depuis l'application mobile.

TABLEAU 4.4 – Statistiques du jeu de données de l'assistant – Source : Auteur

Métrique	Valeur
Questions/réponses brutes	3,914 paires
Documents indexés (chunks)	5,270 segments
Sources uniques	3,898 documents
Sujets (catégories)	13 catégories (étouffement, saignements, brûlures, etc.)
Ensemble d'entraînement nettoyé	3,901 paires
Ensemble de validation nettoyé	409 paires
Changements de labels (audit)	1,841 sur l'entraînement, 165 sur la validation

Le nettoyage des données a utilisé un audit automatisé (`data_quality.py`) qui détecte et corrige les étiquettes erronées assignées d'après les termes de réponse, préserve les étiquettes originales dans `original_topic` pour examen ultérieur, et génère un rapport d'audit détaillé avec traces de modification. L'augmentation ciblée de données a renforcé les sujets faibles (premiers secours généraux, saignements, morsures) en générant 60 exemples synthétiques pour améliorer la couverture et la robustesse.

4.8.3 Entraînement et métriques de performance

L'entraînement est piloté par le script `train_firstaid_chatbot.py`, qui construit le pipeline complet et évalue les performances selon plusieurs métriques.

TABLEAU 4.5 – Performances de l'assistant après entraînement – Source : Auteur

Métrique	Valeur	Cible	Interprétation
Hit Rate / Recall@1	79.71%	$\geq 75\%$	Juste au rang 1
Recall@3	90.95%	-	Juste dans top-3
Recall@5	94.38%	$\geq 90\%$	Juste dans top-5
MRR	0.8627	≥ 0.80	Rang moyen inverse
MAP@5	0.8572	≥ 0.78	Précision moyenne
Evidence F1	0.2990	≥ 0.35	Extraction de span
Fallback Rate	2.69%	$\leq 5\%$	Refus confiance faible
Smoke Tests	14/14	100%	Cas critiques

Analyse des résultats :

- la récupération documentaire est satisfaisante, avec un Recall@5 de 94.38% ;
- le MRR de 0.8627 indique que les documents utiles apparaissent généralement dans les premiers rangs ;
- l'extraction précise de spans reste le point le plus faible, avec un Evidence F1 de 0.2990 ;
- l'écart observé vient surtout de l'appariement entre réponses attendues et passages récupérés ;
- le taux de fallback de 2.69% montre que le mécanisme de confiance intervient dans les cas incertains.

La valeur d'Evidence F1 inférieure à la cible (0.2990 vs 0.35) s'explique principalement par le mode d'appariement entre les réponses de référence et les documents indexés. Le pipeline extrait des passages complets plutôt que des spans précis, ce qui pénalise le score F1 au niveau des tokens. Une amélioration possible consiste à réétiqueter les données avec des spans d'extraction annotés manuellement, ou à intégrer ultérieurement un module extractif supervisé, évalué sur des spans annotés, en remplacement de l'extraction heuristique actuelle.

4.8.4 API et intégration avec l'application mobile

L'assistant est exposé via une API FastAPI dédiée, lancée sur un port séparé (8001 par défaut) pour éviter le conflit avec l'API principale de pharmacies (8000).

Listing 4.2 – Exemple de requête à l'API de l'assistant

```
POST /answer
Content-Type: application/json

{
  "query": "Comment faire face à l'étouffement ?",
  "top_k": 5
}
```

Listing 4.3 – Structure de la réponse de l'assistant

```
{
  "query": "Comment faire face à l'étouffement ?",
  "answer": "Faites-le tousser vigoureusement.",
  "confidence": 0.92,
  "evidence_sources": [docid_1, docid_2],
  "safety_notice": "APPEL D'URGENCE REQUIS",
  "topic": "choking"
}
```

L'intégration côté mobile ajoute un onglet ou un écran dédié à l'assistant, permettant aux utilisateurs de poser des questions de premiers secours tout en accédant à localisateurs de pharmacies de garde. Les requêtes utilisateur sont envoyées à l'API de l'assistant, qui retourne la réponse enrichie avec confiance, sources et avertissements de sécurité.

4.8.5 Tests et validation

L'assistant bénéficie de deux niveaux de validation : des smoke tests couvrant 14 scénarios d'urgence courants, dont l'étouffement, le choc et la perte de conscience, ainsi que trois suites unitaires portant sur la normalisation, la fusion hybride et le calcul de confiance. Comme les réponses sont extraites de passages récupérés, le risque de contenu non ancré est réduit ; il n'est cependant pas présenté comme nul, ce qui justifie la présence d'un score de confiance et de messages de sécurité.

4.9 Captures d'écran

Les captures suivantes présentent les principales interfaces réalisées pour le tableau de bord administrateur et pour l'application mobile.

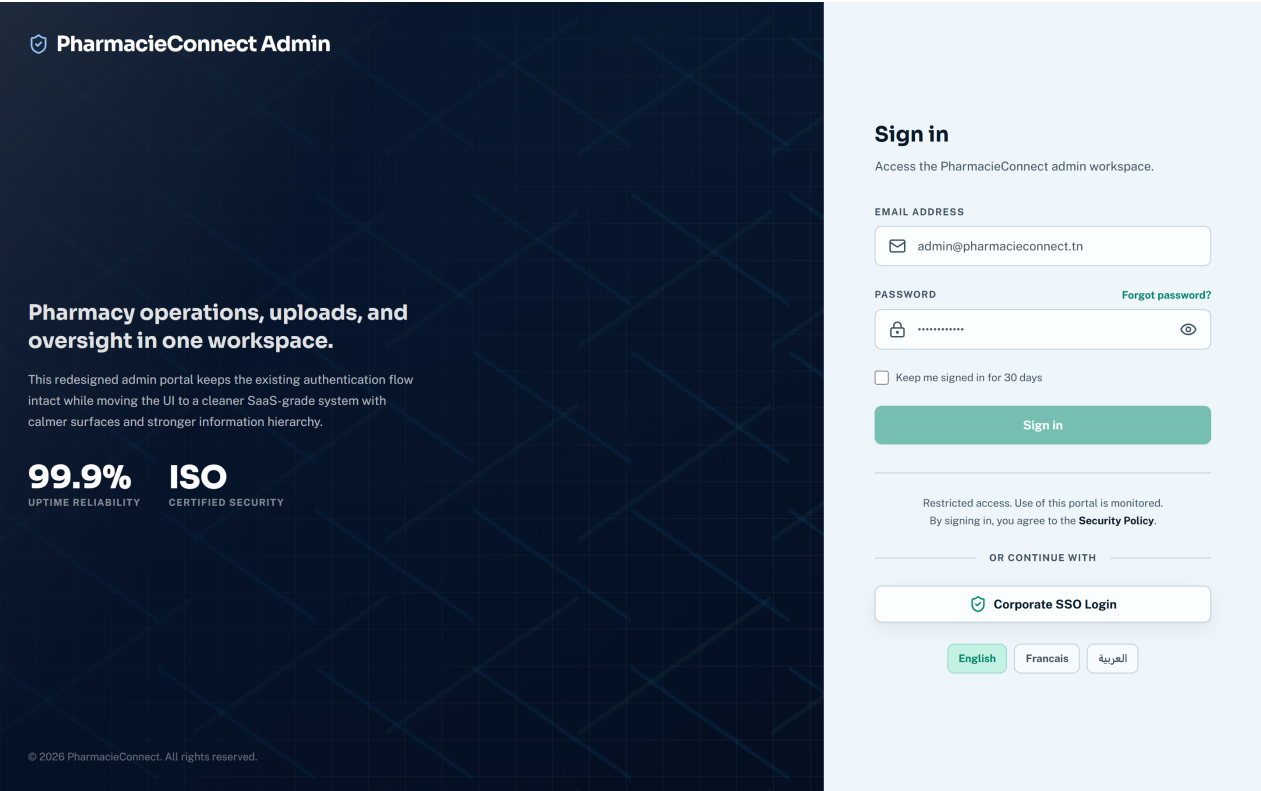


FIGURE 4.2 – Écran de connexion du tableau de bord administrateur – Source : Auteur

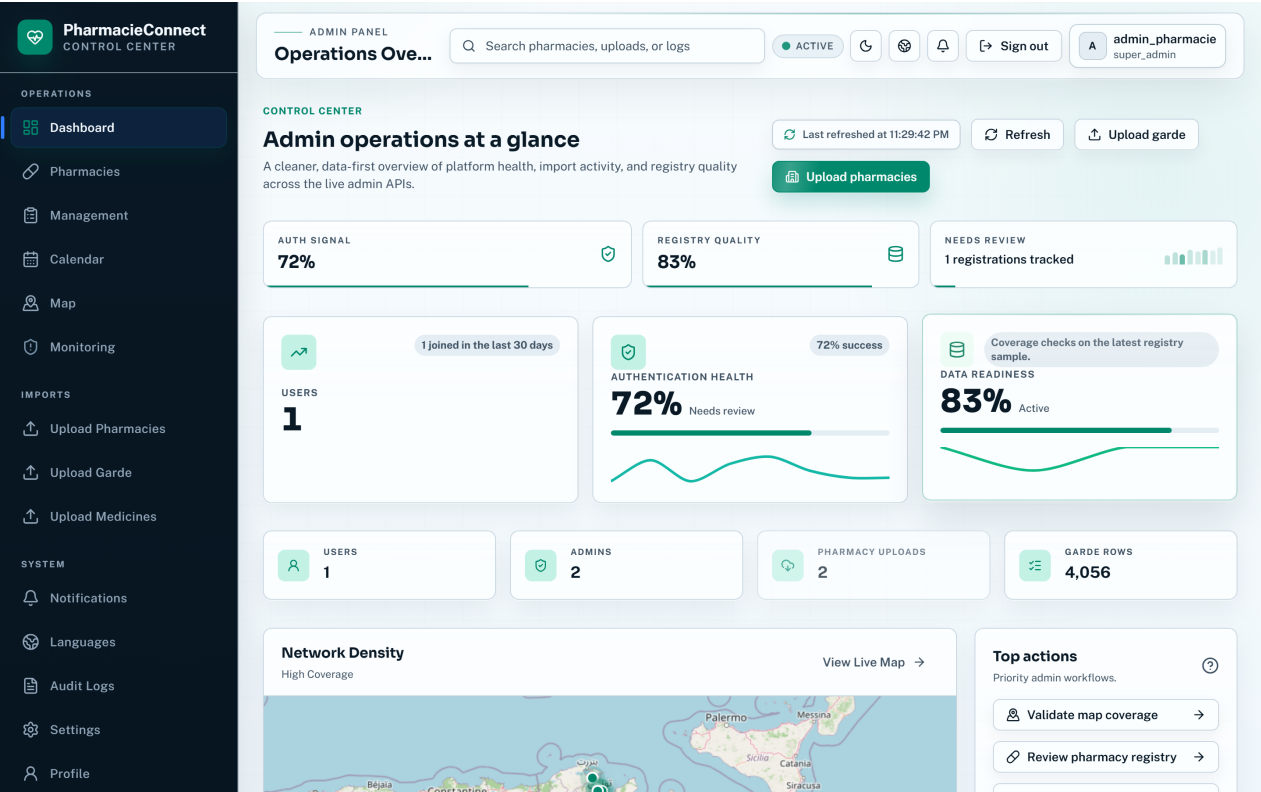


FIGURE 4.3 – Vue du tableau de bord administrateur – Source : Auteur

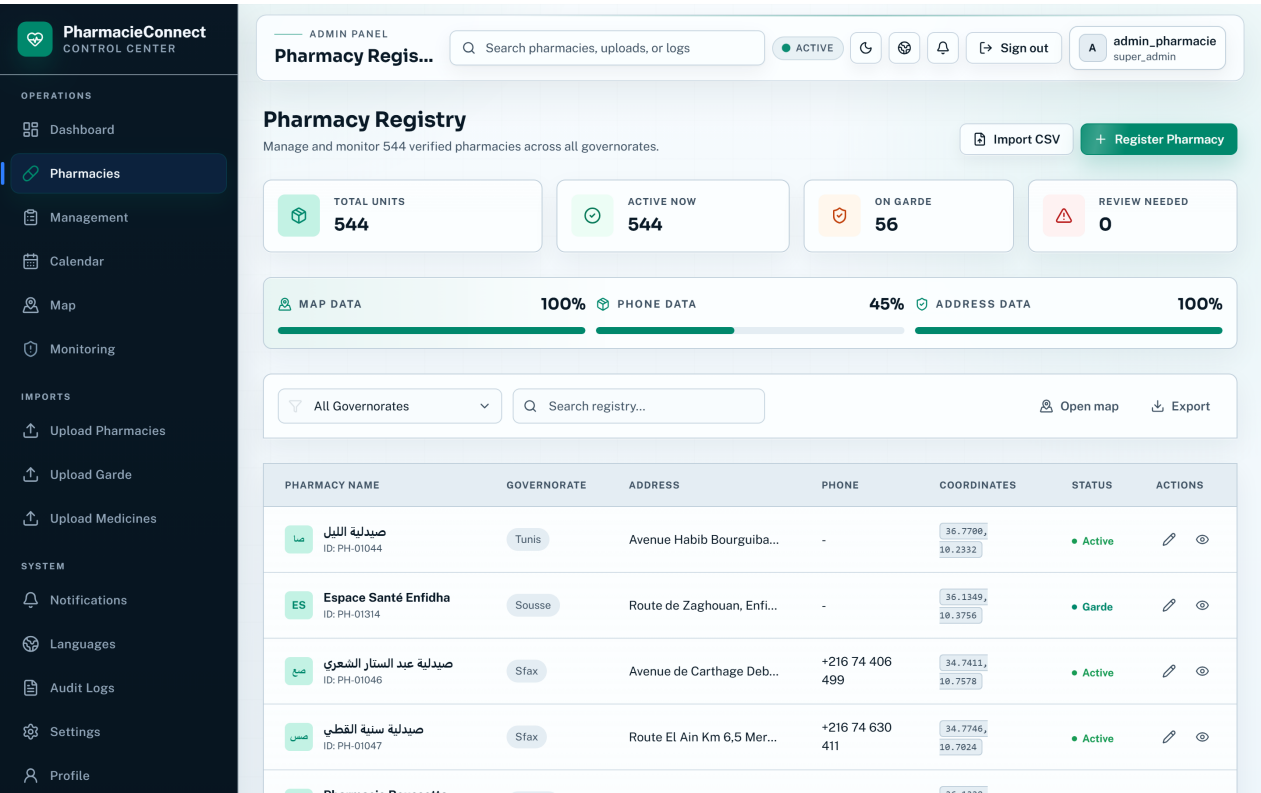


FIGURE 4.4 – Vue du registre des pharmacies – Source : Auteur

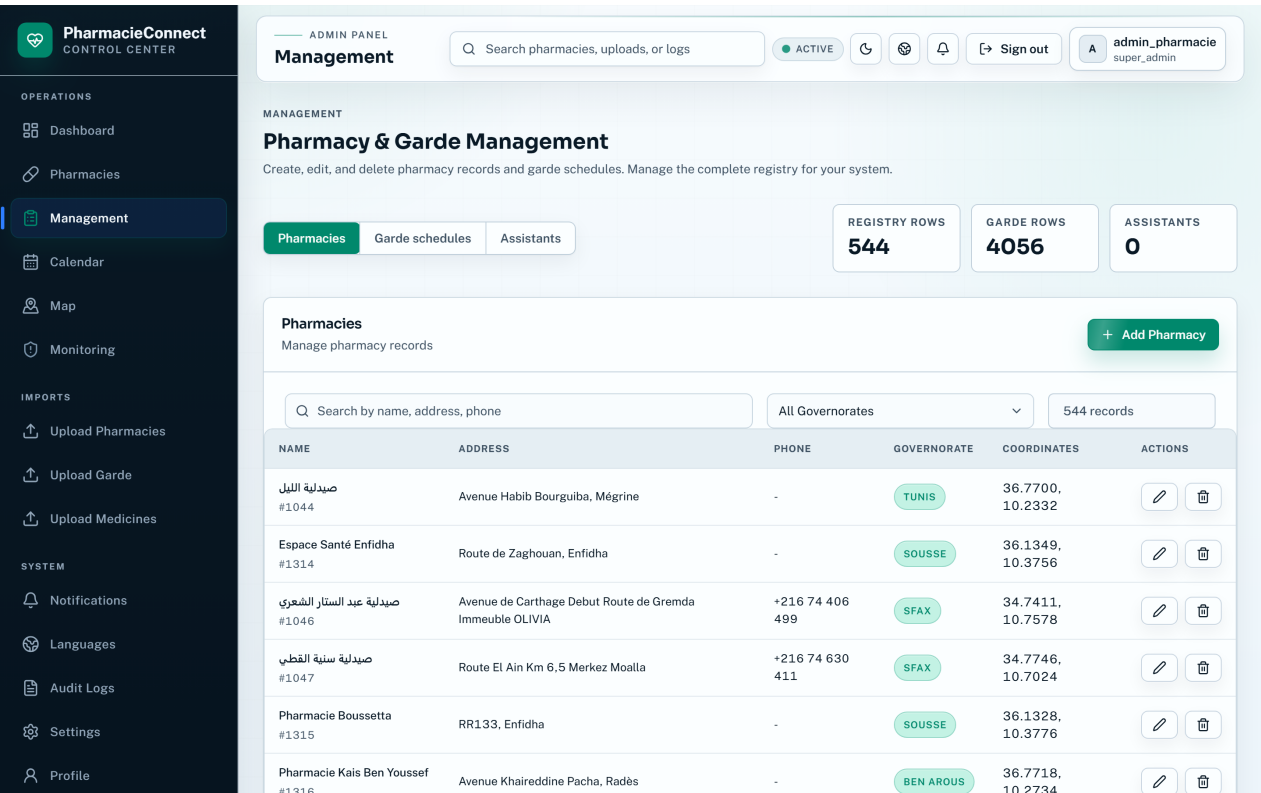


FIGURE 4.5 – Vue de gestion des pharmacies et des gardes – Source : Auteur

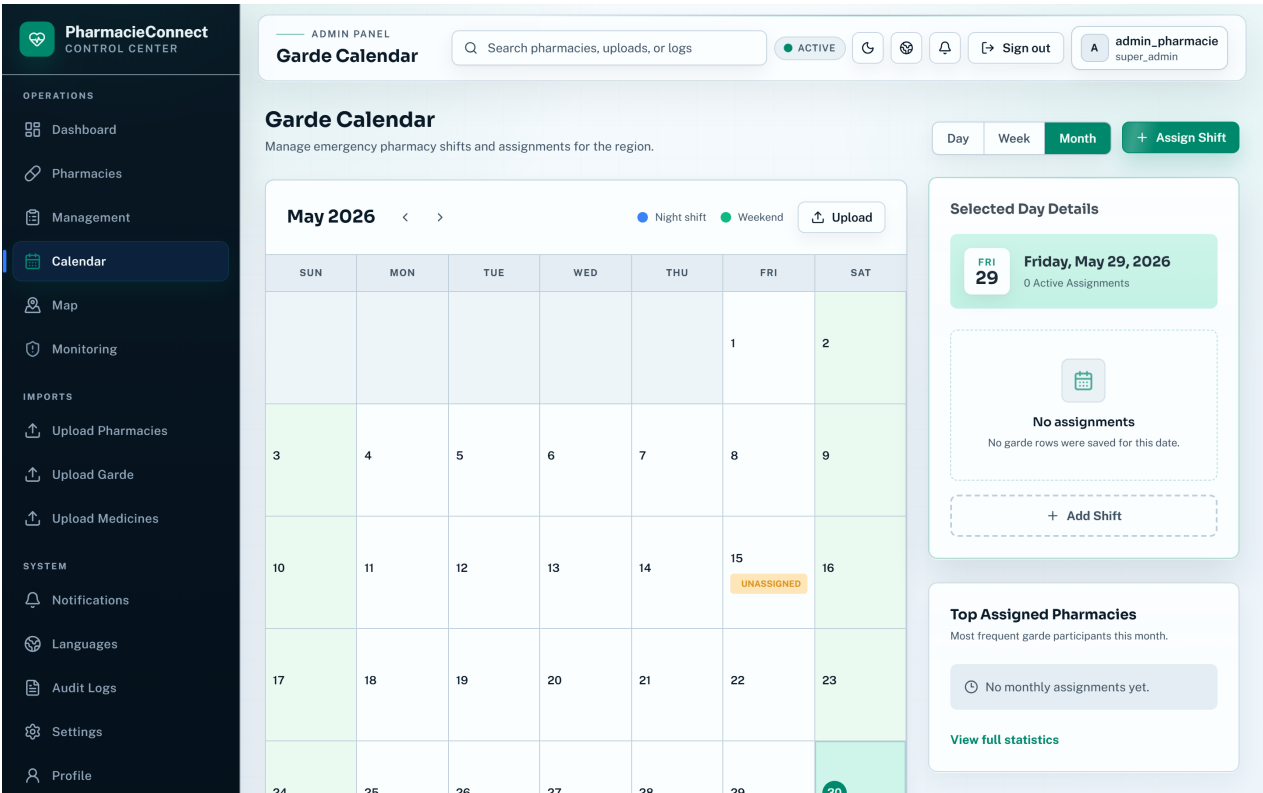


FIGURE 4.6 – Vue du calendrier des gardes – Source : Auteur

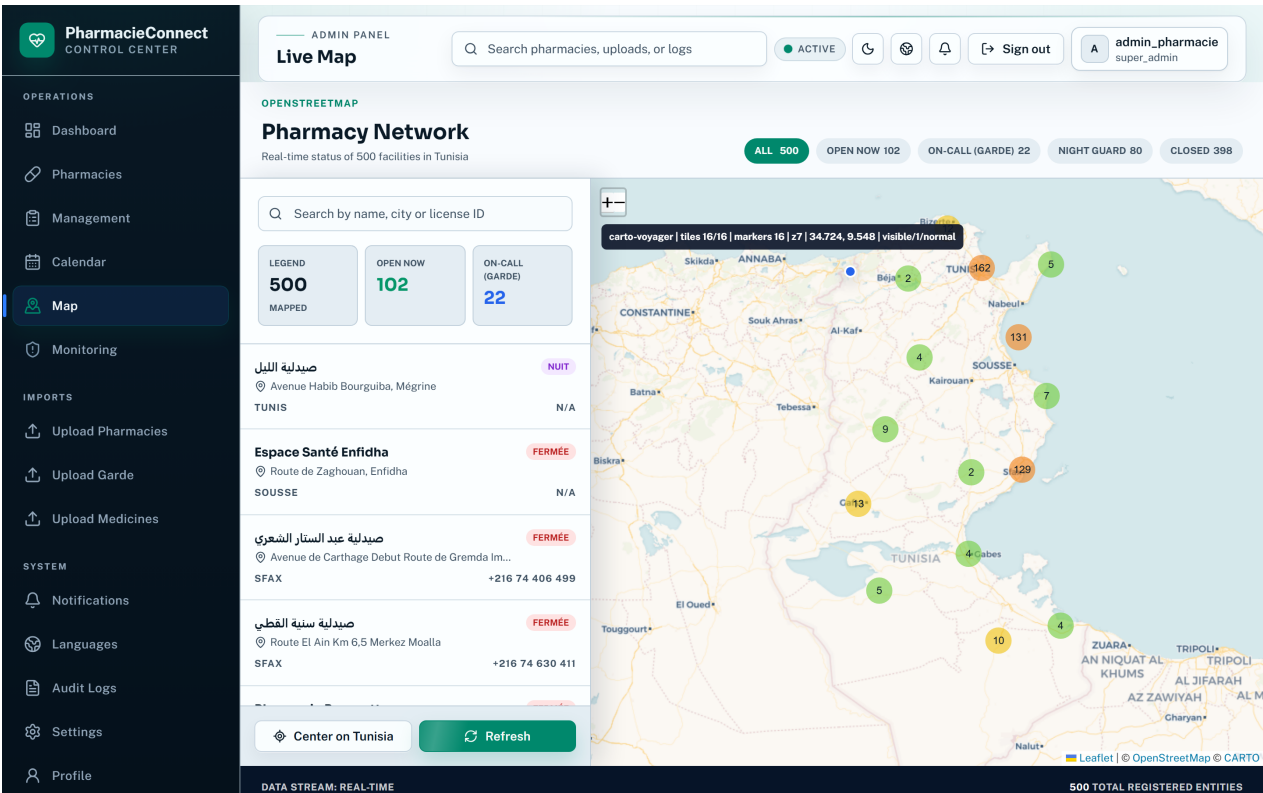


FIGURE 4.7 – Vue cartographique du réseau de pharmacies – Source : Auteur

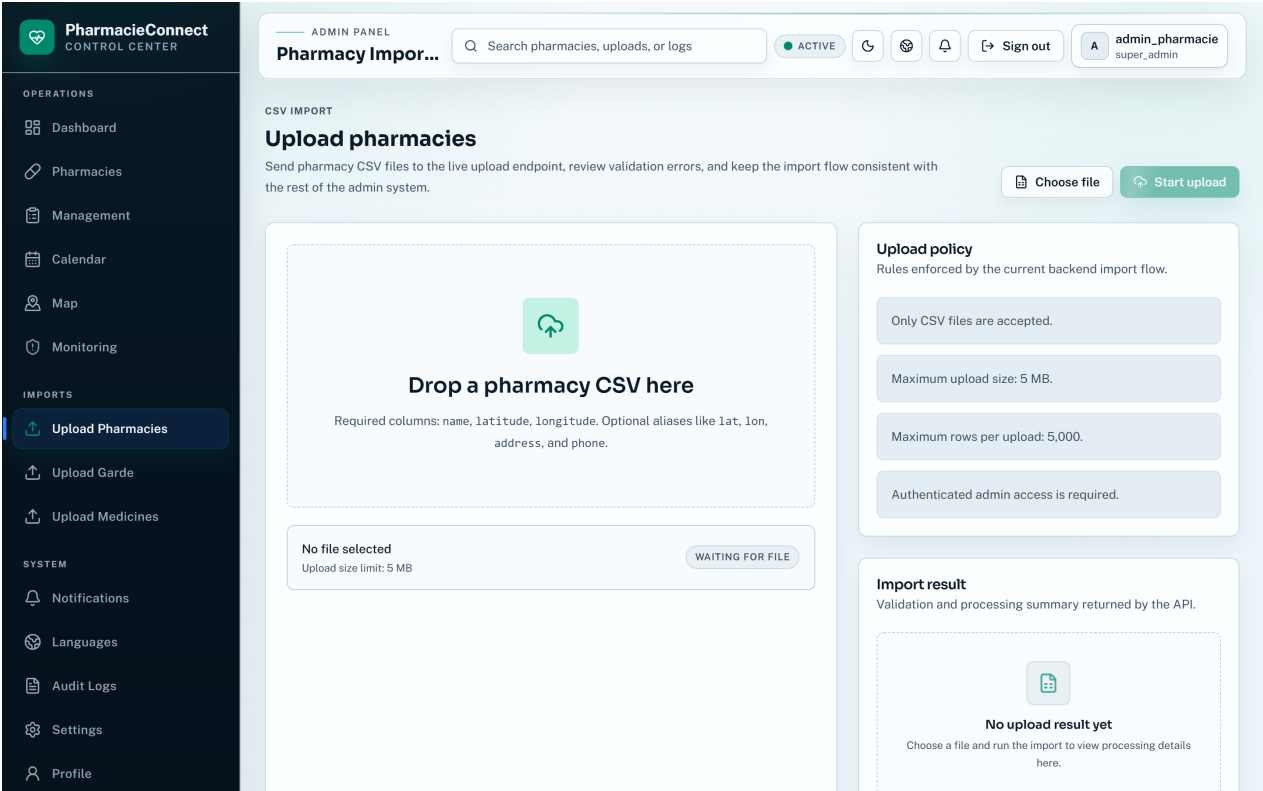


FIGURE 4.8 – Interface d’import des pharmacies – Source : Auteur

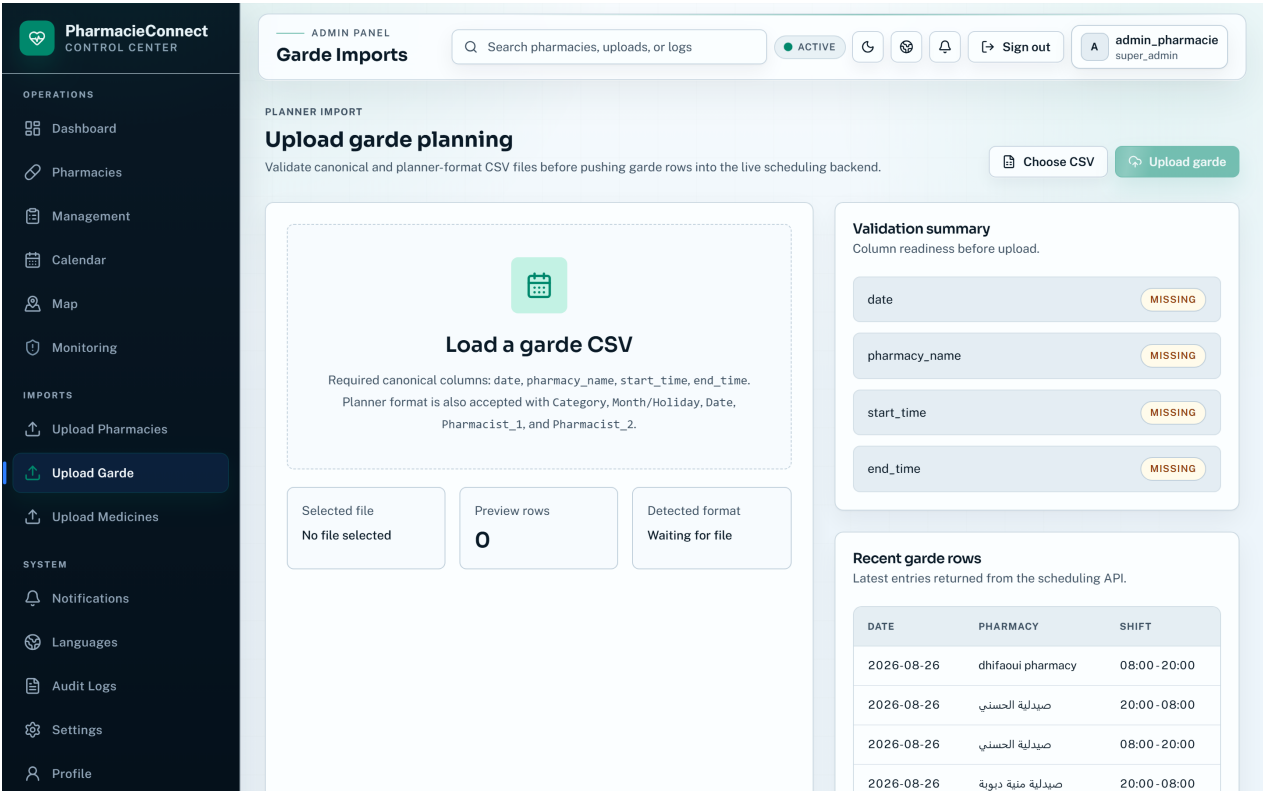


FIGURE 4.9 – Interface d’import du planning de garde – Source : Auteur

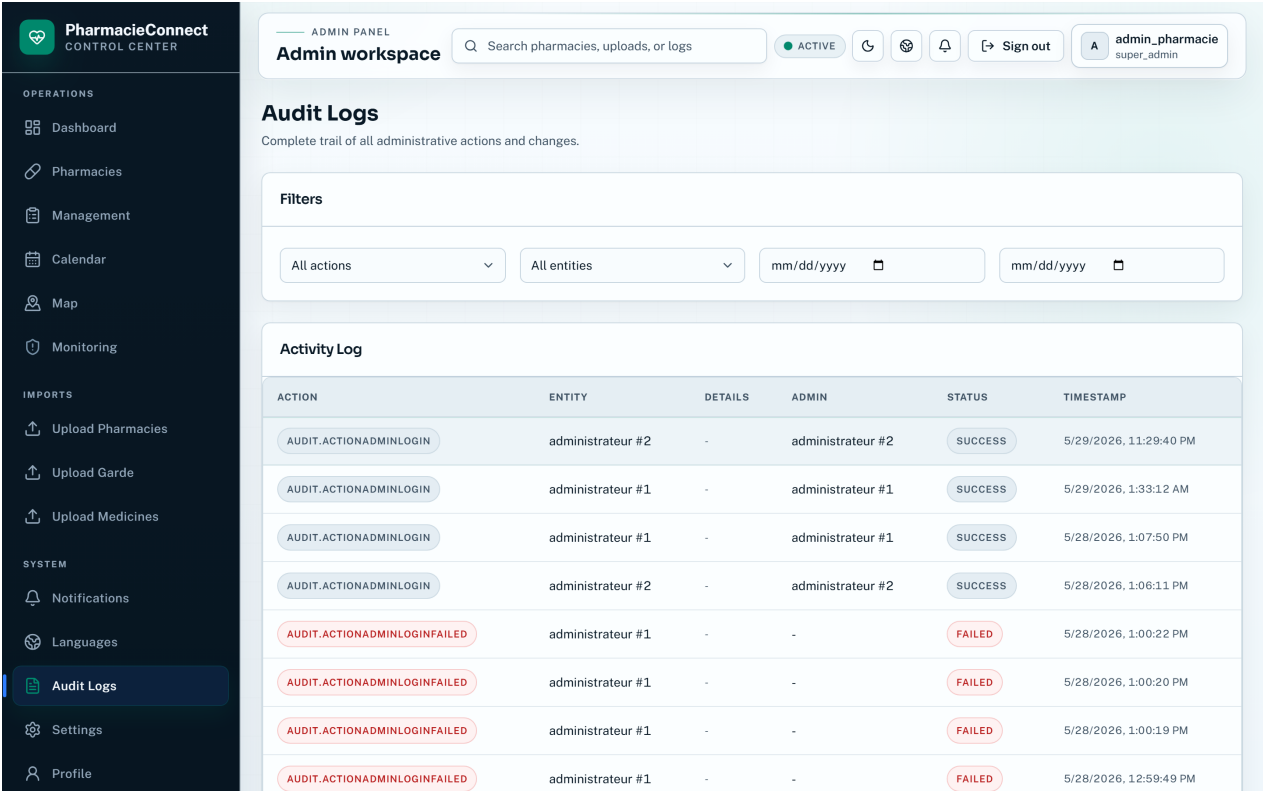


FIGURE 4.10 – Vue des journaux d’audit – Source : Auteur

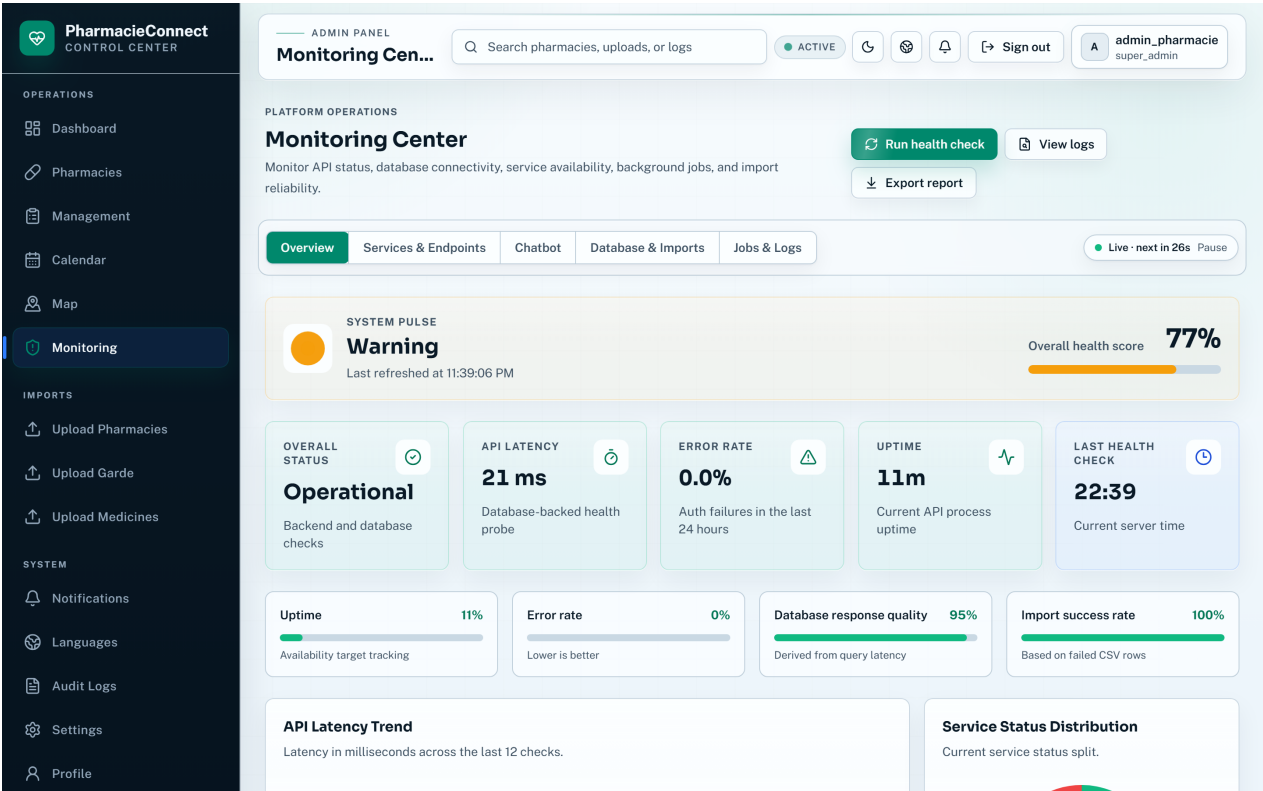


FIGURE 4.11 – Centre de monitoring de la plateforme – Source : Auteur

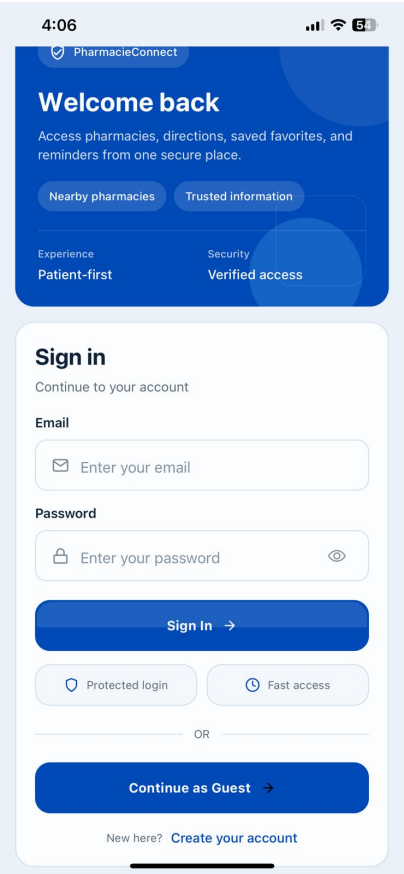


FIGURE 4.12 – Connexion mobile – Source : Auteur

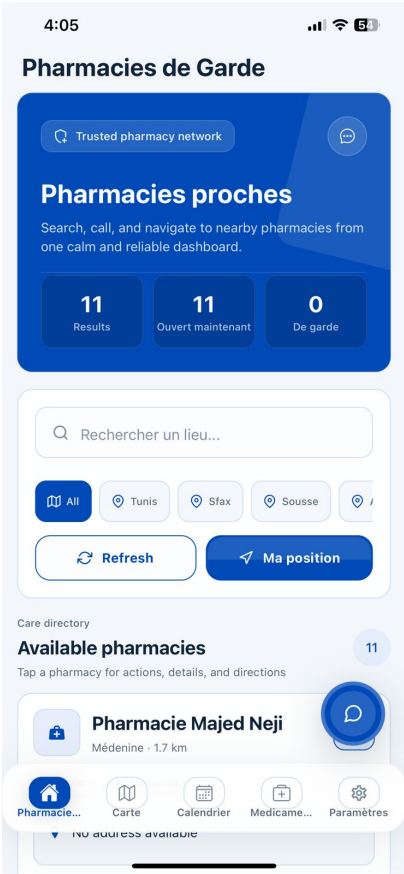


FIGURE 4.13 – Accueil et recherche mobile – Source : Auteur

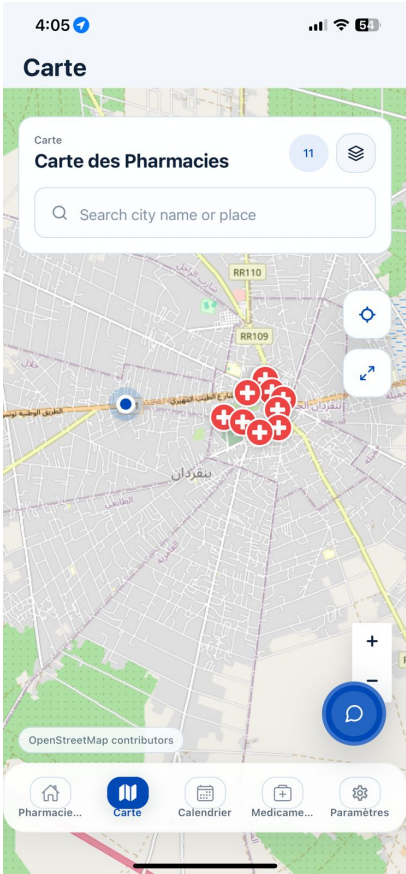


FIGURE 4.14 – Carte mobile des pharmacies – Source : Auteur

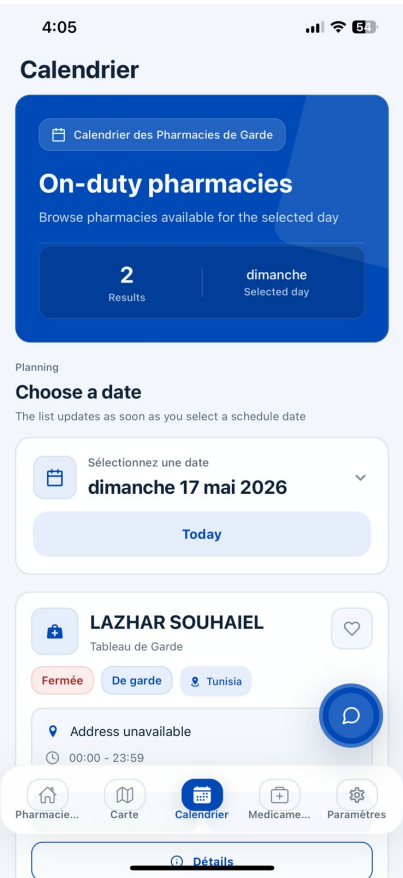


FIGURE 4.15 – Calendrier mobile des gardes – Source : Auteur



FIGURE 4.16 – Catalogue mobile des médicaments – Source : Auteur

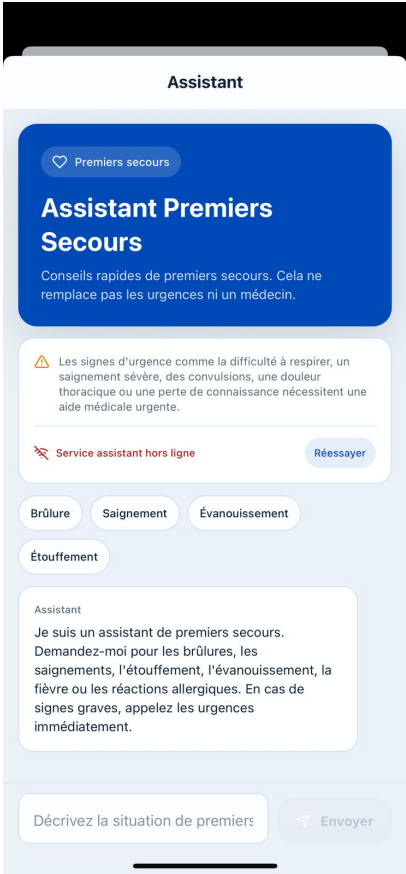


FIGURE 4.17 – Assistant mobile de premiers secours – Source : Auteur

4.10 Intégration API

L’intégration API relie les trois interfaces (tableau de bord, application mobile et assistant) au backend. Le tableau de bord administrateur utilise Axios, tandis que l’application mobile et l’assistant possèdent leurs propres configurations d’appel API. Les trois clients consomment les endpoints selon leurs besoins définis.

TABLEAU 4.6 – Principales intégrations API – Source : Auteur

Client	Endpoints utilisés	Objectif
Application mobile	Pharmacies, gardes, médicaments, authentification	Consultation et parcours citoyen
Tableau de bord admin	Administration, imports, indicateurs, audit	Gestion opérationnelle
Assistant QA	Récupération et classement de documents	Urgences médicales

4.11 Déploiement et configuration

La configuration du backend a été externalisée dans des variables d’environnement. Dans notre cas, cela évite de mélanger le code applicatif avec des valeurs qui changent selon le contexte d’exécution, comme la base de données, les origines autorisées ou les clés de sécurité.

TABLEAU 4.7 – Variables d’environnement principales – Source : Auteur

Variable	Rôle
DATABASE_URL	Chaîne de connexion à la base de données.
SECRET_KEY	Clé utilisée pour signer les jetons JWT.
ALGORITHM	Algorithme de signature des jetons.
ACCESS_TOKEN_EXPIRE_MINUTES	Durée de validité du jeton d’accès.
REFRESH_TOKEN_EXPIRE_DAYS	Durée de validité du jeton de rafraîchissement.
FRONTEND_URL	Origine autorisée pour l’interface web.
TRUSTED_HOSTS	Liste des hôtes autorisés.
ENABLE_REDIS_CACHE	Activation ou désactivation du cache Redis.

Le déploiement en production n’a pas été réalisé dans le cadre de ce PFE en raison des contraintes de temps et de la nature du stage externe. L’architecture retenue est néanmoins compatible avec un déploiement sur des plateformes cloud comme Railway ou Render pour le backend FastAPI, et Vercel pour l’interface d’administration React/Vite. Ces étapes constituent une perspective concrète à court terme.

Pour la version de validation, le projet est exécuté localement de manière contrôlée : le backend FastAPI est lancé avec Uvicorn, l’interface d’administration avec le serveur de développement Vite et l’application mobile avec Expo. Dans cette configuration, l’intégration entre les trois parties peut être vérifiée avant une mise en production. Pour un déploiement final, la même architecture peut être placée derrière un serveur HTTPS, avec PostgreSQL comme base de données persistante et des variables d’environnement séparées pour la production. Les commandes de lancement local sont placées en annexe afin de ne pas alourdir le corps du rapport.

4.12 Tests unitaires, API et manuels

4.12.1 Stratégie de test

La validation de PharmacieConnect a porté sur les parcours qui peuvent avoir un impact direct sur l’usage de la plateforme. Les tests ont concerné l’authentification, l’accès aux routes protégées, la recherche de pharmacies, la consultation des gardes, l’import de fichiers CSV, la cohérence des réponses API et l’intégration mobile. Une attention particulière a été portée aux erreurs d’import, aux droits insuffisants et aux réponses de l’assistant de premiers secours, car ces cas nécessitent des messages clairs et un comportement prudent.

TABLEAU 4.8 – Stratégie de validation – Source : Auteur

Type de test	Objectif
Tests unitaires backend	Vérifier une règle métier isolée dans un service.
Tests API	Vérifier les endpoints, les statuts HTTP, les réponses et les droits d'accès.
Tests d'import	Contrôler les fichiers CSV, les colonnes, les lignes invalides et les bilans d'import.
Tests de sécurité	Vérifier les rôles, la vérification email, les jetons et les accès interdits.
Tests manuels web	Valider les pages d'administration, formulaires, tableaux, cartes et imports.
Tests manuels mobile	Valider la recherche, la carte, le calendrier, les langues, les paramètres et les médicaments.

4.12.2 Résultats de validation automatisée

Une campagne de vérification a été exécutée sur les trois parties du projet le 13 mai 2026. Elle ne remplace pas une recette complète en production, mais elle confirme que les modules principaux compilent ou passent leurs tests automatisés dans l'environnement local.

TABLEAU 4.9 – Résultats de validation exécutés – Source : Auteur

Partie vérifiée	Commande	Résultat observé
Backend : indicateurs, assistants et audit	<code>pytest</code> sur les fichiers ciblés	14 tests réussis, 21 avertissements, durée 4,45 s.
Application mobile	<code>npm test - --runInBand</code>	7 tests réussis sur la recherche mobile par localisation, durée 2,781 s.
Administration web	<code>npm run build</code>	Build Vite terminé avec succès, génération du dossier <code>dist/</code> .
Import médicaments	<code>pytest</code> sur le fichier d'upload médicaments avec limite de temps	Limite connue : exécution interrompue après 20 s sur le premier test ¹ .

Le build de l'administration a produit les artefacts `index.html`, CSS et JavaScript. Vite a signalé un avertissement concernant une taille de bundle JavaScript supérieure à 500 kB après minification. Cet avertissement n'empêche pas le build, mais il indique qu'un découpage dynamique des pages pourrait être ajouté dans une future optimisation.

4.12.3 Tests automatisés du backend

Les tests backend sont écrits avec Pytest. Ils utilisent une base de test afin de ne pas modifier les données de développement. Les fichiers identifiés couvrent notamment :

- `test_analytics_dashboard.py` pour les indicateurs du tableau de bord ;
- `test_assistant_regions.py` pour les restrictions régionales des assistants ;
- `test_audit_logs.py` pour la traçabilité des connexions et des actions ;
- `test_medicine_upload.py` pour l'import et la recherche de médicaments.

1. Ce test dépasse le délai d'exécution en raison du volume de données de test. Ce point constitue une limite connue identifiée pour correction dans une version future.

Les tests d'audit vérifient qu'une connexion administrateur réussie crée une entrée `ADMIN_LOGIN`, qu'un échec crée une entrée `ADMIN_LOGIN_FAILED`, et qu'une inscription crée une entrée `USER_REGISTERED`. Les tests de restrictions régionales contrôlent qu'un assistant ne peut agir que dans son périmètre autorisé. Les tests d'import de médicaments couvrent les colonnes obligatoires, la normalisation des prix, la gestion des doublons par `code_pct` et les messages d'erreur pour les lignes invalides.

4.12.4 Tests API

Les tests API vérifient les routes publiques et protégées. Ils contrôlent les paramètres requis, les limites de pagination, les erreurs de validation, les accès interdits et la cohérence des réponses JSON.

4.12.5 Validation du déploiement local

La version validée du projet est exécutable localement avec trois services séparés : backend FastAPI, administration React/Vite et application mobile Expo. Cette structure reflète l'architecture cible et donne un cadre clair pour vérifier les échanges API avant un déploiement final.

TABLEAU 4.10 – Preuves de lancement et de build – Source : Auteur

Composant	Commande de vérification	État
Backend FastAPI	<code>pytest</code> sur tests ciblés	Tests ciblés réussis.
Administration web	<code>npm run build</code>	Build de production réussi.
Application mobile	<code>npm test -- --runInBand</code>	Tests Jest réussis.
Configuration	Variables d'environnement listées	Paramètres identifiés.

4.12.6 Validation fonctionnelle

Les principaux scénarios fonctionnels vérifiés sont présentés dans le tableau suivant.

TABLEAU 4.11 – Scénarios fonctionnels testés – Source : Auteur

Scénario	Étapes	Résultat attendu
Connexion administrateur	Saisir email et mot de passe, puis valider.	Accès au tableau de bord et stockage des jetons.
Recherche mobile	Saisir un nom, une adresse ou un gouvernorat.	Affichage d'une liste de pharmacies pertinentes.
Recherche par proximité	Autoriser la localisation et lancer la recherche.	Pharmacies triées par distance.
Consultation des gardes	Choisir une date dans le calendrier.	Affichage des pharmacies de garde correspondantes.
Import pharmacies	Charger un fichier CSV valide.	Données importées et bilan affiché.
Import gardes	Charger un planning CSV.	Gardes consultables côté administration et côté mobile.
Recherche médicament	Chercher par nom, DCI ou code PCT.	Résultats pertinents et détail accessible.
Audit logs	Filtrer les actions enregistrées.	Résultats paginés cohérents.

Scénario	Étapes	Résultat attendu
Changement de langue	Passer du français à l'arabe.	Interface traduite avec direction RTL appliquée.

4.13 Difficultés rencontrées et solutions adoptées

Plusieurs difficultés ont été rencontrées pendant la réalisation. La première concerne la qualité des données importées par CSV : certains fichiers peuvent contenir des colonnes manquantes, des doublons ou des valeurs difficiles à exploiter. Pour limiter ce problème, le backend vérifie les champs obligatoires et retourne des erreurs compréhensibles ligne par ligne. La deuxième difficulté concerne la séparation des droits entre administrateur et assistant régional. Cette contrainte a nécessité un contrôle côté serveur afin qu'un compte autorisé ne puisse pas modifier des données hors de son périmètre. Enfin, l'intégration mobile a demandé de stabiliser les appels API, notamment pour la recherche, la carte, le calendrier des gardes et l'assistant de premiers secours.

TABLEAU 4.12 – Difficultés rencontrées et solutions adoptées – Source : Auteur

Difficulté	Solution adoptée
Alignement entre backend, web et mobile	Centralisation des contrats API et séparation des services backend.
Qualité variable des fichiers CSV	Validation des colonnes, contrôle des types, retour des erreurs ligne par ligne et comportement d'upsert pour les médicaments.
Gestion des rôles et des assistants régionaux	Vérification des permissions côté backend et application du périmètre régional dans les services.
Géolocalisation mobile	Demande explicite des permissions, contrôle de l'activation des services de position et recherche par coordonnées.
Maintenabilité du backend	Séparation entre routes, services, modèles, schémas, sécurité et migrations.
Internationalisation mobile	Utilisation de fichiers de traduction et prise en charge de la direction RTL pour l'arabe.

4.14 Conclusion

La réalisation a abouti à une plateforme fonctionnelle composée d'une API sécurisée, d'un tableau de bord administrateur et d'une application mobile. Les tests réalisés ont couvert les parcours essentiels : authentification, recherche de pharmacies, consultation des gardes, imports CSV, gestion des rôles, audit et recherche de médicaments. Des améliorations restent possibles, notamment les tests bout en bout, les tests de charge et la validation sur un volume de données plus important.

Conclusion générale

Bilan

Ce projet a abouti à une plateforme qui réunit, dans une même architecture, la consultation mobile des pharmacies de garde et la gestion administrative des données. **PharmacieConnect** ne se limite pas à afficher des pharmacies : il organise la mise à jour des gardes, le contrôle des imports CSV, la gestion des rôles, la consultation du catalogue de médicaments et la traçabilité des actions sensibles. L'ajout d'un assistant de premiers secours complète le parcours mobile, tout en gardant une approche prudente face aux situations médicales.

Résultats obtenus

Les objectifs principaux fixés au départ ont été couverts. Côté mobile, l'utilisateur peut rechercher une pharmacie, exploiter la géolocalisation, consulter les gardes selon une date donnée et accéder au catalogue de médicaments. Il dispose également d'un assistant de premiers secours qui fournit des réponses encadrées par un score de confiance et des messages de prudence. Côté administration, la plateforme permet de gérer les pharmacies, les plannings de garde, les médicaments, les utilisateurs autorisés, les imports CSV, les indicateurs et les journaux d'audit. Le backend joue le rôle de noyau commun : il centralise les données, applique l'authentification, vérifie les rôles, valide les requêtes et conserve les traces des actions sensibles. L'assistant atteint un Recall@5 de 94.38%, combine une recherche lexicale BM25 avec une recherche sémantique et utilise un mécanisme de confiance pour réduire les réponses incertaines.

Sur le plan technique, ce travail a mobilisé des compétences complémentaires : conception d'API REST, développement backend avec FastAPI, modélisation avec SQLAlchemy, développement web avec React et Vite, développement mobile avec Expo et React Native, validation de fichiers CSV, authentification JWT, journalisation, tests automatisés et mise en place d'un pipeline de récupération d'informations. La réalisation a aussi confirmé l'importance d'une séparation claire entre routes, services, modèles, schémas, interfaces clientes et modules spécialisés. Cette organisation a facilité les tests, les corrections et l'évolution progressive des fonctionnalités.

Limites

Certaines limites doivent toutefois être soulignées. Le catalogue des médicaments ne décrit pas encore le stock réel de chaque pharmacie ; il constitue plutôt une base de consultation. Les gardes restent dépendantes de la qualité des fichiers importés et de la régularité des mises à jour administratives. L'assistant obtient de bons résultats en récupération, mais la sélection précise des passages de réponse reste perfectible, comme le montre l'Evidence F1 de 0.2990 pour une cible de 0.35. Les tests bout en bout, les tests de charge et la validation sur un ensemble plus large d'appareils

mobiles devront également être renforcés avant une exploitation à grande échelle.

Apport personnel

Sur le plan personnel, ce projet m’a permis de consolider mes compétences en développement full-stack et d’approfondir des sujets plus spécifiques comme la sécurité applicative, les droits d’accès, la qualité des données importées et la récupération d’informations. La gestion simultanée d’un backend, d’une interface web, d’une application mobile et d’un assistant m’a amené à raisonner en contrats entre composants : quelles données circulent, qui peut les modifier, comment les erreurs sont remontées et comment les actions sensibles restent vérifiables. Les difficultés rencontrées ont renforcé l’importance d’une démarche itérative, fondée sur les tests et sur la correction progressive des limites identifiées.

Perspectives

Les perspectives d’amélioration peuvent être regroupées autour des axes suivants :

- ajouter la disponibilité des médicaments par pharmacie afin de rapprocher le catalogue d’un service opérationnel ;
- permettre aux pharmacies de mettre à jour certaines informations sous validation administrative ;
- envoyer des notifications lorsque les gardes changent ou lorsqu’une pharmacie proche est disponible ;
- améliorer la recherche par synonymes, fautes de frappe et noms commerciaux proches ;
- renforcer l’extraction des passages de réponse de l’assistant par un reclassement plus fin ou un réétiquetage des données ;
- étendre la base de connaissance de l’assistant à d’autres sujets utiles dans le contexte pharmaceutique ;
- améliorer les capacités multilingues de l’assistant, notamment pour l’arabe et l’anglais ;
- intégrer des tests bout en bout couvrant l’application mobile, le tableau de bord, l’API et l’assistant ;
- préparer un déploiement de production avec supervision, sauvegardes et suivi des performances ;
- enrichir les indicateurs pour mesurer les recherches, la couverture territoriale et l’usage de l’assistant ;
- créer une interface d’administration dédiée aux données d’entraînement et aux métriques de l’assistant.

En conclusion, PharmacieConnect constitue une base fonctionnelle et évolutive pour la gestion des pharmacies de garde. Le projet reste perfectible, mais il démontre l’intérêt d’une architecture organisée autour d’un backend commun, d’un tableau de bord contrôlé, d’une application mobile orientée citoyen et d’un assistant fondé sur la récupération d’informations. Cette combinaison permet de répondre à la fois aux besoins de consultation, de gestion, de traçabilité et d’assistance.

Bibliographie

- [1] Conseil National de l'Ordre des Pharmaciens de Tunisie, "Site officiel du cnopt." <https://cnopt.tn/>, 2026. Consulté le 13 mai 2026.
- [2] Med.tn, "Pharmacie de garde." <https://www.med.tn/pharmacie/garde>, 2026. Consulté le 13 mai 2026.
- [3] Google, "Google maps." <https://www.google.com/maps>, 2026. Service consulté le 13 mai 2026.
- [4] OpenStreetMap Foundation, "Openstreetmap." <https://www.openstreetmap.org/>, 2026. Service consulté le 13 mai 2026.
- [5] Conseil national de l'ordre des pharmaciens de Tunisie, "Apothicare app." <https://apps.apple.com/tn/app/apothicare/id6760540666>, 2026. Page App Store consultée le 13 mai 2026.
- [6] Conseil National de l'Ordre des Pharmaciens de Tunisie, "Tableau de garde." <https://cnopt.tn/tableau-de-garde/>, 2026. Page officielle consultée le 26 mai 2026.
- [7] K. Schwaber and J. Sutherland, "The scrum guide." <https://scrumguides.org/>, 2020. Guide consulté le 13 mai 2026.
- [8] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley, 2005.
- [9] Conseil National de l'Ordre des Pharmaciens de Tunisie, "Conseil régional de l'ordre des pharmaciens de tunis – missions principales." <https://cnopt.tn/sud-gabes/>, 2026. Page officielle consultée le 26 mai 2026.
- [10] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [11] Atlassian, "Coordinate and monitor work with sprints." <https://support.atlassian.com/jira-software-cloud/docs/coordinate-and-monitor-work-with-sprints/>, 2026. Documentation officielle consultée le 26 mai 2026.
- [12] Atlassian, "How to use bitbucket cloud." <https://www.atlassian.com/software/bitbucket/guides>, 2026. Documentation officielle consultée le 26 mai 2026.
- [13] Atlassian, "Gitflow workflow." <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>, 2026. Tutoriel officiel consulté le 26 mai 2026.
- [14] Atlassian, "Connect jira to bitbucket." <https://support.atlassian.com/jira-cloud-administration/docs/connect-jira-cloud-to-bitbucket/>, 2026. Documentation officielle consultée le 26 mai 2026.
- [15] FastAPI, "Fastapi documentation." <https://fastapi.tiangolo.com/>, 2026. Documentation officielle consultée le 12 mai 2026.

- [16] M. O. Source, “React documentation.” <https://react.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [17] Vite, “Vite documentation.” <https://vite.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [18] Expo, “Expo documentation.” <https://docs.expo.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [19] M. O. Source, “React native documentation.” <https://reactnative.dev/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [20] SQLAlchemy, “Sqlalchemy documentation.” <https://docs.sqlalchemy.org/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [21] O. Foundation, “Owasp top 10 web application security risks.” <https://owasp.org/www-project-top-ten/>, 2026. Référence générale consultée le 12 mai 2026.
- [22] M. Jones, J. Bradley, and N. Sakimura, “Json web token.” <https://www.rfc-editor.org/rfc/rfc7519>, 2015. RFC 7519.
- [23] Passlib, “passlib.hash.bcrypt – bcrypt.” <https://passlib.readthedocs.io/en/stable/lib/passlib.hash.bcrypt.html>, 2026. Documentation officielle consultée le 26 mai 2026.
- [24] Pydantic, “Pydantic documentation.” <https://docs.pydantic.dev/latest/>, 2026. Documentation officielle consultée le 26 mai 2026.
- [25] P. G. D. Group, “Postgresql documentation.” <https://www.postgresql.org/docs/>, 2026. Documentation officielle consultée le 12 mai 2026.
- [26] Axios Collective, “Axios documentation.” <https://axios-http.com/docs/intro>, 2026. Documentation officielle consultée le 26 mai 2026.
- [27] Tailwind Labs, “Tailwind css documentation.” <https://tailwindcss.com/docs/installation/using-vite>, 2026. Documentation officielle consultée le 26 mai 2026.
- [28] React Navigation, “React navigation documentation.” <https://reactnavigation.org/docs/getting-started/>, 2026. Documentation officielle consultée le 26 mai 2026.
- [29] i18next, “i18next documentation.” <https://www.i18next.com/overview/getting-started>, 2026. Documentation officielle consultée le 26 mai 2026.
- [30] OpenStreetMap Foundation, “Nominatim usage policy.” <https://operations.osmfoundation.org/policies/nominatim/>, 2026. Politique d’utilisation consultée le 13 mai 2026.
- [31] S. E. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, and M. Gatford, “Okapi at trec-3,” in *Proceedings of The Third Text REtrieval Conference, TREC 1994*, pp. 109–126, National Institute of Standards and Technology, 1994. Consulté le 26 mai 2026.
- [32] Sentence Transformers, “all-minilm-l6-v2 model card.” <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, 2026. Modèle Hugging Face consulté le 26 mai 2026.
- [33] Pytest, “Pytest documentation.” <https://docs.pytest.org/>, 2026. Documentation consultée le 13 mai 2026.
- [34] M. O. Source, “Jest documentation.” <https://jestjs.io/docs/getting-started>, 2026. Documentation consultée le 13 mai 2026.
- [35] S. S. Muna, R. I. Salvi, M. R. Mushfique, and A. Abrar, “Firstaidqa : A synthetic dataset for first aid and emergency response in low-connectivity settings.” <https://arxiv.org/abs/2511.01289>, 2025. arXiv :2511.01289, consulté le 26 mai 2026.